Eksamensnotater

DATA1500 Databaser

Av Jørgen Skontorp

Til fritt bruk under CC BY-NC-SA 4.0 lisens

FAQ fra eksamen:

Med Torunn sitt fasitforslag

Diskuter fordeler og ulemper med relasjonsdatabaser kontra NoSQL-databaser.

Fordeler relasjonsdatabaser:

- Sjekk for integritet og referentiell integritet gir kvalitet på data.
- Støtte for transaksjoner.
- Autentisering og kontroll av rettigheter ved brukt av systemet.
- Logging av data og muligheter for recovery om noe går galt.
- Spørringer kan få ut alle mulige kombinasjoner informasjon fra tabellene i systemet
- SQL er et felles språk.
- Følger ACID egenskaper.

Ulemper relasjonsdatabaser:

- Store ressurskrevende dataprogrammer
- Data lagret i mange tabeller fordelt på ulike steder på lagringsmediet gir trege systemer.
- All kontroll av rettigheter og alt tilleggsarbeid systemet utfører gjør det å hente data tregere.

NoSQL databasesystemer er en sekkebetegnelse på mange ulike databasesystemer som deles i følgende grupper:

- Nøkkel/verdi-databaser
- Dokumentdatabaser
- Kolonnedatabaser

Felles for systemene er at datamengdene ofte er store, krav til skalering for mange brukere, krav til hurtig levering av data. Noen NoSQL databasesystemer av egner seg til sanntids informasjonsdeling feks sosiale medier. Det kan være sanntidssystemer som spilldata som replikeres til brukerne, det kan være store datamengder som registreres automatisk og bare aggregeringsdata som skal lagres varig. Systemene kan være dokumentbaserte og er laget for at informasjonsinnhenting skal gå fort, dette kan være i XML eller i JSON format. Databasen ligger ofte på flere maskiner og data blir replikert og fragmentert. Data i NoSQL systemer er ofte mindre

strukturerte enn de vi har i relasjonsdatabaser. Ulemper er at man må sette seg inn i nye APIer og lære seg nye måter å få data ut av databasene.

Gi eksempler på når du vil benytte relasjonsdatabaser og når du synes det er hensiktsmessig å benytte en annen type database.

Hvis integritet og sikkerhet er viktig som f.eks i et banksystem ville det være nødvendig å benytte relasjonsdatabaser.

Facebook-data er helt annerledes. Her er datamengdene enorme og krav til skalering og hurtighet veldig store. I et slikt system ville det være nyttig å benytte en NoSQL-løsning.

Hva er en trigger, hvordan fungerer en trigger og hva benyttes triggere til? Gi eksempler på hva en trigger kan benyttes til.

Et (lite) program som blir utført i databasen

- Kjøres automatisk som følge av en hendelse i databasen
- Mulige hendelser: INSERT, UPDATE, DELETE
- Vi sier at triggeren avfyres

Triggere er små programsnutter på databaseserveren som settes i gang før eller etter en hendelse som insert, update og delete. Programmene kan kjøres før endringer skrives til databasen eller etter at det er skrevet endringer til systemet. En trigger benyttes for operasjoner som skal gjøres uten unntak. Triggere kan f.eks benyttes til å validere inndata eller til å loggføre etter at en operasjon har funnet sted.

Vi har to hovedtyper av triggere BEFORE triggere og AFTER triggere.
En BEFORE trigger fyres av før oppdateringen skjer i databasen og kan bennyttes til:

- - Automatisk formatering av verdier
- - Automatisk datostempling av verdier
- - Kontroll ved forretningsregler
- - Finmasket tilgangskontroll.

En AFTER trigger fyres av etter oppdateringen har skjedd i databasen og brukes typisk til å:

- Automatisk oppdatere data i andre tabeller
- Automatisk loggføre endringer
- Oppdatere avledede data i andre tabeller.

Hvordan lage en trigger?

```
CREATE TRIGGER triggernavn
  [BEFORE | AFTER]
  [INSERT | UPDATE | DELETE] ON tabellnavn
  [FOR EACH ROW]
BEGIN
```

```
 setninger
END
```

Ett eksempel på en kreasjon av en trigger

```
CREATE TRIGGER tr_Ansatt
     BEFORE UPDATE ON Ansatt
     FOR EACH ROW
     BEGIN
     set new.Fornavn = UPPER(new.Fornavn);
END
```

Kan benyttes til loggføring, automatisk følgeoppdatering og håndtering av forretningsregler.

XML er format som det er støtte for i de fleste browsere. Hva er XML og hvorfor er det egnet som format på web?

XML er et metaspråk, et språk for å definere nye språk. Det står for eXtensible Markup Language, og man kan definere egne tagger og strukter i XML-tekstfiler. XML defineres ofte av en DTD-fil eller Document Type Definition som definerer strukturen av taggene, om taggene er obligatoriske og hva slags datatype de er. XML Schema kan også brukes som Dokument Definisjon for XML-filer, som er et rikere DTD med flere funksjoner som DTD ikke støtte, som feks Støtte for nøkler, flere datatyper, og støtte for XML navnerom. XML schema bruker også XML syntaks i motsetning til DTD.

Det er godt egnet som format på web fordi det er rene tekstfiler, som gjør at det i utgangspunktet er programvare og system-uavhengig. HTML som er det vanligste formatet å presentere innhold på web er definert i XML. Siden XML er rene tekstfiler er de ideele for datautveksling.

Siden XML krever hiarkisk oppbygging av dataene, er det mulig å veldefinere strukturen til websiden. XML lar deg også programmere mot taggene og få mange typer data ut ved bruk av XSLT.

Når benyttes XML-filer? Hva er fordeler og ulemper med XML-filer kontra databaser?
XML er spesielt velegnet for datautveksling, der databaser er best for datalagring. Dette kan for eksempel være data som benyttes av andre applikasjoner (web). XML har en hierarkisk sortering av data og har en mer fleksibel struktur enn en database. XML burde ikke benyttes til lagring av store mengder data. XML er egnet som format på web fordi det er rene tekstfiler som gjør at det i utgangspunktet er uavhengig av programvare eller system. HTML som er det vanligste formatet å presentere innhold på web er definert i XML. Siden XML krever hierarkisk oppbygging av dataene, er det mulig å veldefinere strukturen til websiden. I XML lager vi en tre-struktur av databasetabellen: Ett element for hele tabellen, ett element for hver rad og ett element for hvert felt

XML kan med fordel benyttes når:

- Data skal deles med andre
- Data skal leses, men ikke slettes og endres
- Det ikke skal gjøres kompliserte spørringer mot dataene
- Data ikke trenger tilgangskontroll
- Data skal utveksles mellom ulike systemer

- Data skal lagres uavhengig av datasystemer
- Data skal leses og presenteres på web-sider

Hva er forskjellen på tett og ikke-tett indeksering? Hvilke felt kan det indekseres ikke- tett på? Hvorfor er flernivå-indeksering ofte standard indekseringsmetode i et databasehåndteringssystem?

En indeks er en hjelpetabell som inneholder et tabellfelt og en peker til hvor dataene befinner seg. En tett indeks har en peker til hver eneste post på fila, mens en ikke-tett indeks bare har pekere til starten av datablokker.

En ikke-tett indeks kan bare brukes på det feltet fila er fysisk sortert på. Har også ikke-tette indekser i flere-nivå indekser da indeksblokkene her er sortert. Fler-nivå indeksering vil gi antall lesinger på disk=antall nivåer i indeksen. Dette er mye raskere for store datamengder enn vanlige indekser siden indeksfilen ved store datamengder blir stor og krever mange lesinger.

Forklar hva en indeks er, hvordan man oppretter indekser med SQL og hvilke kolonner det er hensiktsmessig å indeksere.

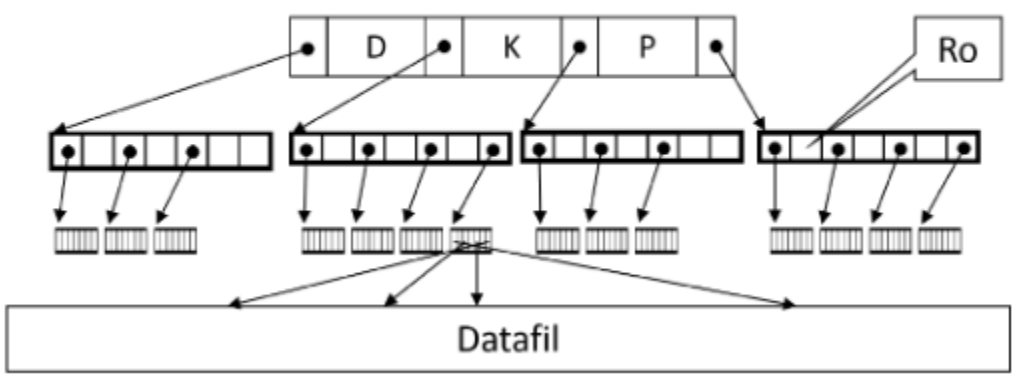
Indekser benyttes for å få raskere oppslag i databasen. Man indekserer på et eller flere felt i en tabell. Indeksen blir en ekstra tabell som bare inneholder indeksfeltet og en peker til hvor posten ligger på disk.

```
CREATE INDEX NavnIdx ON (Kunde(Etternavn))
```

Det er hensiktsmessig å indeksere på kolonner det ofte søkes på som navn, varenr, telefon. Positivt med utfyllende svar, fint om flernivå-indekser nevnes.

Forklar hva en flernivå-indeks er og hvorfor dette gir færre disklesinger enn andre typer indekser.

Antall lese-operasjoner:	**B+-trær**
1	
2	
3	
4	



Anta vi skal finne post der Etternavn = «Roheim».

1. Les inn rot-blokken.
2. Søk i rot-blokken: Her skal vi følge pekeren helt til høyre.
3. Gjenta søk på mellomliggende nivåer ...
4. ... Les inn riktig blokk fra datafilen.

Antall lesinger er antall nivåer i treet + lesingen av dataene som skal finnes. Med vanlige indekser vil antall lesinger i indeksen være 2 opphøyd i tall som gir antall poster. For store datamengder blir det langt mer enn 4 lesinger i indeksen.

Hva er kravene for at tabeller er på 3NF?

Kravene for at tabeller er på 3NF er at de er på første, andre og tredje normalform.
For at en tabell skal kunne være i 1. normalform, kan den ikke inneholde ikke-atomære verdier eller repeterende grupper.

- **Ikke-atomære verdier:** Verdier som inneholder flere delverdier
- **Repeterende grupper:** At det finnes mer enn en verdi i krysset mellom rad og kolonne i tabellen.

For at en tabell skal kunne være i 2. normalform, må alle verdier være fullt funksjonelt avhengig av primærnøkkelen.

- **Full funksjonell avhengighet:** Betyr at man ikke har noen partielle avhengigheter
- **Partiell avhengighet:** Betyr at en verdi er avhengig av kun deler av primærnøkkelen

For at en tabell skal kunne være i 3. normalform, kan man ikke ha noen transitive avhengigheter

- **Transitiv avhengighet:** Betyr at en verdi er avhengig av en annen ikke-primærnøkkel-verdi

For at en tabell skal kunne være i Boyse-Codd normalform, må alle determinanter være kandidatnøkler

- **Determinant**: Attributt eller gruppe attributter som en annen attributt er fullt funksjonelt avhengig av
- **Supernøkkel**: Et eller flerer attributter som sammen identifiserer en entitet unikt
- **Kandidatnøkkel**: Minimalistiske supernøkler, dvs at man kan fjerne et attributt fra en supernøkkel og så er det fortsatt unikt

Hvilke typiske feilsituasjoner kan oppstå i et databasesystem og hvilke mekanismer er innført for å håndtere disse?
Typiske feilsituasjoner ved et system som er i kontinuerlig drift:

- Strømbrudd
- Disk kræsj
- Applikasjonsfeil (prosesser som stopper, køer som går fulle, etc.)
- Etc.

Mekanismer for å håndtere dette:

- Kommandoer for transaksjonshåndtering (rollback og commit)
- Innføring av loggføring av transaksjoner etter hvert som de blir utført for å kunne rekonstruere systemet til tilstanden før feilen oppstod
- Recovery. Gjennomoppbygging av transaksjonsloggen (redo, undo)

Beskriv noen typiske problemer som kan oppstå i et databasesystem som har flere samtidige brukere.

Typiske problemer:

- Når flere brukere har aksess til den samme databasen, kan de typisk ødelegge for hverandre dvs. det kan oppstå konflikter
- Lesing ikke noe problem alle kan lese samtidig
- Problemer ved skriving til databasen
- Granularitet: finkornethet
  - OK å skrive til ulike databaser?
  - Ok å skrive til ulike tabeller?
  - Ok å skrive til ulike poster?
  - Ok å skrive til ulike felt?

Bør nevne minst ett av de to typiske problemsituasjoner der man har mange samtidige transaksjoner med korte beskrivelser:

- Tapt oppdatering: En transaksjon overskriver en annen
- Inkonsistent innhenting/analyse: Henter inn data som er under endring
- Angret oppdatering: En transaksjon jobber på data som ikke er skrevet permanent til databasen

Gi eksempler på hvilke problemer kan oppstå i et databasesystem som ikke har mekanismer som håndterer samtidige brukere.

- Tapt oppdatering
- Inkonsistent analyse
- Se bok kap. 10.3.4 for konkrete eksempler

Gjør rede for noen teknikker som et databasehåndteringssystem bruker for å håndtere mange, samtidige brukere.

- Låser: leselåser som kan deles, skrivelåser som bare en kan sette av gangen.
- Tofaselåsing for å tilfredsstille ACID transaksjoner.
- Loggføring for å kunne rekonstruere databasen i feilsituasjoner.
- Tidsstempler, ventegrafer for å håndtere evt. Vranglåser

Hva er serialiserbare forløp?

Vil ha serialiserbarhet: Resultat uavhengig av rekkefølge på transaksjonene

Et forløp er en bestemt rekkefølge for hvordan alle deloperasjonene i et sett av samtidige transaksjoner utføres. Et sekvensielt forløp er en rekkefølge der hver enkelt transaksjon utføres en etter en (dvs. alle deloperasjoner i hver transaksjon før neste påbegynnes). Dette er ikke effektivt og man ønsker større grad av samtidighet, men opprettholde korrekthet i utførelsen. Det betyr her som om de enkelte transaksjonene hadde blitt utført sekvensielt. Et serialiserbart forløp har samme effekt som et serialiserbart forløp.

Hva er en vranglås? Nevn eksempler på hvordan en vranglås kan løses opp

To typer lås:

- Delt lås ved lesing andre som skal lese kan komme inn
- Eksklusiv lås ved skriving, andre transaksjoner får ikke tilgang

Ved vranglås har man en evig vent situasjon der transaksjon T1 venter på en lås transaksjon T2 har og omvendt. Systemet henger.

Vis gjerne med eksempel med to transaksjoner som ender i en situasjon som beskrevet over eller beskriv kort "The dining philosophers" Eksempler på hvordan en vranglås kan løses opp:

- Unngå vranglås
  - Alle låser settes før transaksjon startes
  - Nummerere alle ressurser og kreve at de innhentes i stigende rekkefølge
  - Detektere vranglås og fjerne denne

Kan gjøres ved algoritmer, kan tegne ressursallokeringsgraf og se om løkke
Hvis vranglås, finn offer og ta rollback på denne transaksjonen

«Strutse-algoritme» – tro at det ikke oppstår, ta ROLLBACK på en transaksjon hvis time-out krever tidsstempling

Hvilke SQL-kommandoer benytter databaseadministrator for å lage en bruker, opprette en database og gi brukeren rettigheter til databasen?

```
CREATE DATABASE
     CREATE USER
     GRANT (rettigheter) ON (objekt) TO (user)
```

Forklar hva som skiller en relasjonsdatabase fra et datavarehus

Tabeller på 3NF koblet sammen ved hjelp av nøkler. OLTP-system (Online Transaction Processing) omhandler driften av et bestemt system. Bruker ofte korte og enkle transaksjoner og spørringer. Et datavarehus er et sentralt datalager, der man samler alle relevante data virksomheten trenger for å få et helhetlig blikk. Typisk vil datavarehuset inneholde sammenstilte data fra mange kildesystemer og en viktig oppgave er å sikre tilstrekkelig datakvalitet slik at man får et pålitelig underlag for analyser, rapporter og beslutninger. Et datavarehus er et OLAP-system (Online Analytical Processing). OLAP omhandler historisk data og arkivdata. Bruker ofte kompliserte spørringer og få transaksjoner.

Datavarehus:

- Mange ulike datakilder
- Extract, Transform, Load
- Stjerneskjema
- Kuberepresentasjon
- OLAP system

- Brukes til aggregerte data
- Benyttes til beslutningsstøtte og analyse

Forklar hvordan man kan benytte en hashfunksjon til rask lesing av dataposter fra disk.

En hash funksjon tar en input og returnerer et svar. Funksjonen gir stor spredning. Ved å benytte en hashfunskjon på f.eks et kundenr kan man benytte svaret fra funksjonen som adressen til der det lagres. Dvs. at man beregner hvor det er lagret og slipper å ha mange lesinger på disk

Hva vil det si at en transaksjon tilfredsstiller ACID?

En transaksjon må følge ACID egenskaper:

- **Atomicity**: Alle eller ingen av deloperasjonene i en transaksjon må fullføres.
- **Consistency**: En transaksjon fører databasen fra en lovlig tilstand til en annen lovlig tilstand.
- **Isolation**: Effekt av transaksjoner under utførelse skal ikke kunne observeres av andre transaksjoner.
- **Durability**: Effekt av fullførte (comitted) transaksjoner lagres i databasen og skal ikke gå tapt på grunn av feil.

Notater

Bonus: https://www.studocu.com/no/document/oslomet-storbyuniversitetet/databaser/ovingsark-teori-databaser/4623897

Begreper

Work in progress...

DBHS – Databasehåndteringssystem

Minne

Disk

DBA – Databaseadministrator

Metadata – data om data

Ulike navn på deler av tabell...

Entitet

Attributt

Blokk

Redundans

Null-merker

Heap-filer - Ikke-sorterte filer. Se filstrukturer.

CRUD – Create, read, update, delete. De fire viktigste operasjonene mot databasetabeller. INSERT, SELECT, UPDATE og DELETE.

Unionskompatibel

OID - Objektidentifikator

Nested spørring/delspørring

Gjenstår:

fullføre begrepsliste, SQL-setninger

mer om hashing?, mer om relasjoner/forhold?

Koblinger

Databaser består som regel av tabeller. Skriver ikke særlig mer om det her, men se presentasjonene av kap 2-4 og SQL-spørringer.

Vi kan utføre spørringer mot flere tabeller og på den måten koble de sammen. Når vi skal koble i SQL er det viktig å spesifisere hvilke tabeller vi henter kolonner fra. Det gjør vi ved å skrive tabell1.kolonne. Vi kan også lage alias for å slippe å skrive hele tabellnavnet. Feks:

SELECT t1.Navn, t2.Lønn
FROM tabell1 AS t1, tabell2 AS t2

Når vi kobler to eller flere tabeller kan vi få kryssprodukt eller ulike koblinger, slik som likekobling.

(se også SQL-oppslagsverket for ulike joins)

Kryssprodukt (kartesisk produkt)
SELECT *
FROM Kunde, Ordre

KNr	Navn
1	Per
2	Ola

OrdreNr	KNr	AnsNr
1	1	21
2	2	21
3	2	28

2 × 3 = 6 !

KNr	Navn	OrdreNr	KNr	AnsNr
1	Per	1	1	21
1	Per	2	2	21
1	Per	3	2	28
2	Ola	1	1	21
2	Ola	2	2	21
2	Ola	3	2	28

Eksempelet over viser et kryssprodukt. Denne spørringen er meningsløs og fører til mye redundans. Likevel viser den hvordan det fungerer. Ved å velge alle fra disse to tabellene får vi et kryssprodukt. Det gir 2 kolonner + 3 kolonner og 2 rader x 3 rader. Vi får derfor 5 kolonner og 6 rader. Som illustrasjonen viser får vi masse unødvendig redundans. Dette gjelder for alle spørringer som henter data fra flere tabeller uten restriksjoner. Resultatet vil gi produktet av kolonnene i hver av tabellene som antall rader i tabellen. Kryssprodukt vil gi store resultater som krever mye plass. Derfor er det vanlig å legge inn noen restriksjoner i spørringene og å bruke andre typer koblinger.

I eksempelet er det en-til-mange relasjon mellom kunde og ordre. Det gjør at hvis vi kobler riktig så får vi like mange resultater som det er ordre. Da bruker vi ***indre koblinger***.

Likekobling

SELECT *
FROM Ordre, Kunde
WHERE Ordre.KNr = Kunde.KNr

KNr	Navn
1	Per
2	Ola

OrdreNr	KNr	AnsNr
1	1	21
2	2	21
3	2	28

KNr	Navn	OrdreNr	KNr	AnsNr
1	Per	1	1	21
2	Ola	2	2	21
2	Ola	3	2	28

- Tar som regel kun med koblingskolonne(r) fra én av tabellene. De inneholder jo de samme verdiene.

Likekobling er et eksempel på en indre kobling. To kolonner er ***unionskompatible*** hvis de har samme domene, dvs. at de inneholder samme slags verdier. I dette tilfellet er KNr unionskompatible. Likekoblinger blir oftest koblet opp mot fremmednøkkel, men må ikke gjøre det så lenge man har samme verdier i tabellene som kan brukes til kobling. Det må være samme datatype og samme slags verdier. De trenger ikke ha samme navn.

Istedenfor å bruke WHERE x = x kan man bruke INNER JOIN. Det ser slik ut:

SELECT OrdreNr, OrdreDato, O.KNr, Etternavn
FROM Ordre AS O INNER JOIN Kunde AS K
ON O.KNr = K.KNr

Egenkoblinger

Egenkobling er å koble en tabell med seg selv. Det gjør vi når vi skal sammenlikne noe med hverandre som står i samme tabell. DBHS lager på en måte to kopier av samme tabell som man kan bruke til å sammenlikne med. Det brukes hvis vi har fremmednøkkel som peker på primærnøkkel i samme tabell.

AnsNr	Fornavn	Etternavn	Leder
1	Georg	Barth	2
2	Gunnlaug	Angeltveit	
3	Morgan	Dalland	7
6	Vilde	Aksnes	8
7	Henriette	Brobakken	2
8	Synøve	Bakketun	2

Ytre koblinger

Hvis vi har to tabeller som for hver kunde viser alle ordre som hører til kunden, ønsker vi gjerne å vise samtlige brukere, også de uten ordre. Hvis vi bruker likekobling vil den kun vise kunder med «match» i ordretabellen. Da bruker vi ytre koblinger.

Vi har venstre og høyre koblinger (LEFT/RIGHT OUTER JOIN). En høyre kobling mellom t1 og t2 vil gi alle radkombinasjonene som er med i en likekobling + en rad for hver rad i den høyre tabellen som ikke er med i den høyre tabellen (t2). Det samme gjelder venstre kobling, bare da i motsatt rekkefølge. Venstre og høyre viser til rekkefølgen vi velger å skrive tabellene i SQL-setningen. Disse blir da de samme:

```
SELECT * FROM t1 LEFT OUTER JOIN t2
ON t1.x = t2.y;


SELECT * FROM t2 RIGHT OUTER JOIN t1
ON t1.x = t2.y;
```

KNr	Navn
1	Per
2	Ola
3	Lise

OrdreNr	KNr	AnsNr
1008	1	25
1009	2	25
1010	1	28

KNr	Navn	OrdreNr	KNr	AnsNr
1	Per	1008	1	25
2	Ola	1009	2	25
1	Per	1010	1	28
3	Lise			

Illustrasjonen viser et eksempel på hvordan en venstre ytre kobling vil se ut.

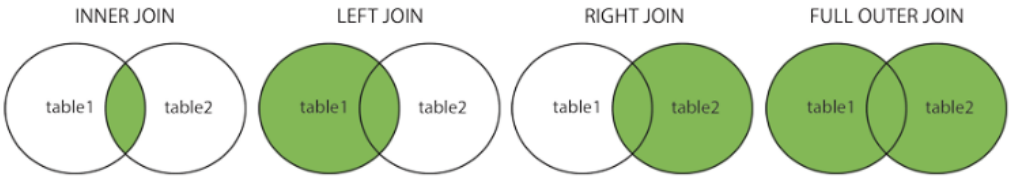
Snitt, union og differanse

Union (UNION), snitt (INTERSECT) og differanse (EXCEPT) er operasjoner man også kan bruke i koblinger hvor vi har unionkompatible tabeller. To spørreresultater er unionskompatible hvis de har like mange kolonner og kolonnene er parvis unionskompatible (samme datatype).

UNION tilsvarer OR, INTERSECT tilsvarer AND og EXCEPT tilsvarer AND NOT

Full ytre kobling

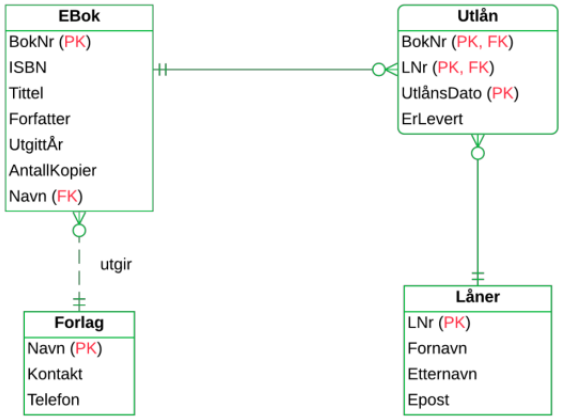
Returnerer alle poster når det er en match i enten venstre eller høyre tabell.



<u>OBS:</u> MySQL støtter ikke fulle ytre koblinger, INTERSECT og EXCEPT

ER-modellering

Datamodellering handler om å strukturere en database. Det er lurt å gjøre visuelt for å få et bedre oversiktsbilde over hva man trenger i databasen og hvordan den ser ut. ER (entity relationship) er et visuelt modellspråk.



Til høyre er en illustrasjon av et ER-diagram (dette burde ha navn på alle relasjonene). Den består av:

Entiteter – Hver boks som tilsvarer en tabell i databasen

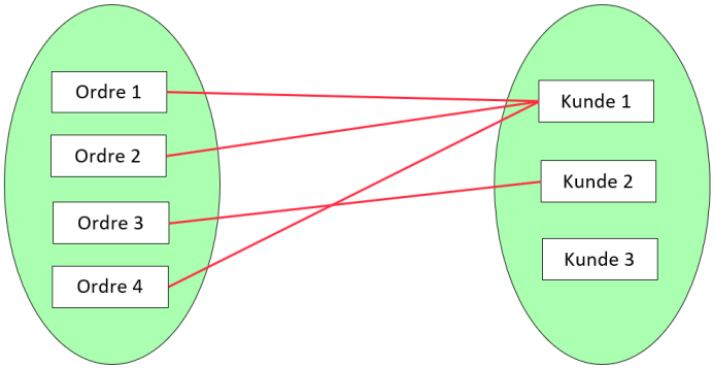
Attributter – De forskjellige tingene som står inni entitetene. Disse tilsvarer kolonner i en tabell.

Indentifikatorer – PK (ofte understreket) som er primærnøkkelen i tabellen. Altså en unik verdi som identifiserer hver rad i tabellen.

Forhold – Linjene mellom entitetene. Dette er relasjonen mellom tabellene som viser hvordan de er koblet sammen ved hjelp av fremmednøkler (FK)

Når man modellerer et databasesystem utfører man en ***kravanalyse***. Dette gjøres ved å gjøre faktainnsamling. Hva trenger vi å ha med i systemet? Når vi ER-modellerer er det viktig å lese oppgaveteksten ***<u>NØYE</u>*** for å analysere hva systemet skal ha med.

Kardinalitet



Kardinalitet sier noe om forholdet/relasjonen mellom tabeller. Om det er en-til-en, mange-til-en eller mange-til-mange. Det noterer vi ved en strek eller kråkefot på relasjonen, som i illustrasjonen over. Kardinalitet uttrykker hvor mange entitetsforekomster av entitet A som kan eller må knyttes til en gitt forekomst av entitet B og omvendt.

I en en-til-en relasjon vil det hvor hver A-forekomst maksimalt være knytta til en B-forekomst og omvendt. Feks: En avdeling kan ha maks en avdelingsleder og en ansatt kan være leder for maksimalt en avdeling.

I en en-til-mange relasjon kan en A-forekomst være knyttet til mange B-forekomster, men hver B-forekomst vil være knyttet til maksimalt en A-forekomst. Feks. En ansatt jobber kun på en avdeling, men en avdeling kan ha mange ansatte.

I en mange-til-mange relasjon kan en A-forekomst være knyttet til mange B-forekomster og omvendt. Feks: En ansatt kan delta i mange prosjekter og et prosjekt kan bestå av mange ansatte.

Vi kan også bruke ***minimumsskranker*** og ***maksimumsskranker*** i en relasjon. De tegner vi innenfor «hovedrelasjonen» (se bilde av ER-diagram). Vanligvis er skranker 1 eller 0. Vi kan ha null-mange, en-

mange, en-en. Feks: Det kan være ansatte som ikke jobber med noe prosjekt, men et prosjekt må ha minst en ansatt knyttet til seg. Ved en-en så vil det si at det *må* være en, altså minst 1, maks 1.

OBS: Det er vanligvis null-mange i en mange-relasjon, fordi vi må ta hensyn til at når vi oppretter en tabell, eller legger inn en rad så kan den ikke være tilknyttet noen enda. Vi bruker nesten bare en-mange når det er absolutt nødvendig at det skal være minst en!

Mer om relasjoner/forhold??

Egenforhold

Det er mulig for entiteter å ha en relasjon til seg selv. Da får man en relasjonslinje som går fra entiteten til entiteten selv. Det kalles egenforhold. Selve relasjonslinja ser ut som en vanlig relasjon. Eksempler på dette er entiteten Person hvor personer kan ha forskjellige relasjoner til andre personer, som feks kollegaer.

Svake entiteter

En svak entitet er en entitet som ikke kan eksistere uten en annen entitet. Feks: Hvis vi har entitetene kino og kinosal. En kinosal kan ikke eksistere uten en kino. Da blir det en svak entitet. Disse noteres ofte med stiplede linjer (vi gjør ikke dette i våre modelleringer)

Identifikator (primærnøkkel)

Alle entiteter må ha en identifikator, i form av en (eller kombinasjonen av flere) primærnøkler. Noen har naturlige identifikatorer, feks fødselsnummer, registreringsnummer, studentnr osv. Hvis det ikke finnes noen kan man opprette en egen (løpenummer). Svake entiteter har ofte en sammensetning av flere

Abstraksjon

Av og til er det ikke nødvendig å vise alle detaljene i en datamodell. For eksempel å droppe å vise svake entiteter, attributter, osv. Abstraksjon er nyttig når man ikke trenger å se detaljene, men vil ha et oversiktsbilde.

<u>Noen abstraksjonsmekanismer:</u>

- Spesialisering / generalisering
  - Selger er en spesialisering av Ansatt.
  - Ansatt er en generalisering av Selger.
- Aggregering / dekomponering
  - Sykkel er en aggregering av hjul, ramme, sete, styre og pedaler.
  - Dekomponering av sykkel gir …
- Kategorisering / instansiering
  - Ansatt er en kategorisering av Hans, Lise, …

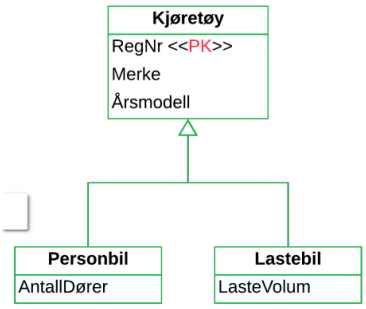
- Lise er en instans av Ansatt.
- Jmf. type og forekomst

Hode/linje-mønsteret (master/detail)

Mange registreringsskjemaer og rapporter er delt inn i to deler: et «hode» og et antall «linjer». Eksempler:

- Et bestillingsskjema har ordredato og kundenummer i hodet, og så linjer som hver beskriver varenummer og antall enheter.
- En timeliste har ansattnummer og tidsperiode i hodet, og så et antall linjer som spesifiserer prosjektnummer og antall timer.
- Et vitnemål har studentnummer, studiestart og studieslutt i hodet, og så et antall linjer med kurskode, eksamensdato og karakter.

Subtyper og hiearkier



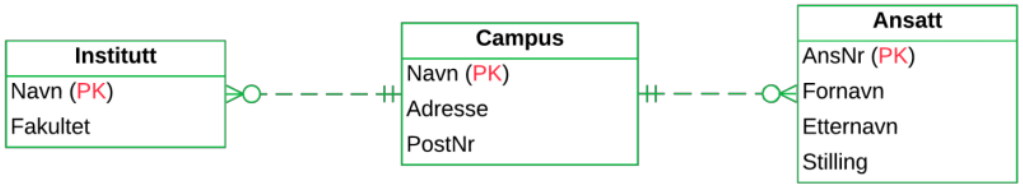
En subtype er en spesialisering av en annen entitet. I illustrasjonen er Personbil og Lastebil subtyper av Kjøretøy. Kjøretøy er da supertypen til de to andre. Både Personbil og Lastebil er spesielle former for Kjøretøy. Subtyper arver egenskapene til supertypen og kan i tillegg ha ekstra attributter og relasjoner. Subtypen er en entitet som representerer en delmengde av forekomstene til supertypen. Uten disse subtypene ville vi hatt alle attributtene inne i Kjøretøy. For å finne ut om noe er en subtype kan vi si «er-en». Personbil er et Kjøretøy. Men en ordre er ikke en kunde og kan derfor ikke være subtyper.

Modelleringsfeller

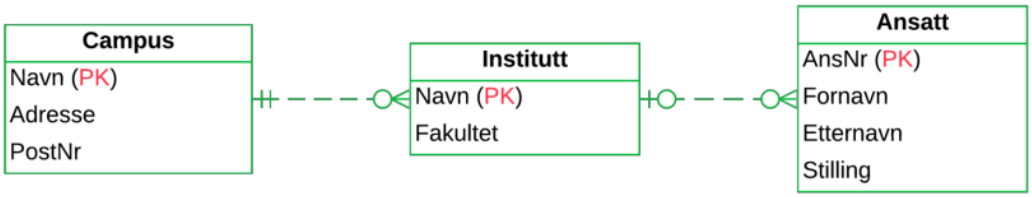
Problemer med dårlig tabelldesign viser seg når databasen tas i bruk. Det finnes ulike feller man kan gå i. Ett av dem er ***redundans***. Hvis vi ikke deler opp en tabell riktig kan vi ende med mye redundans, altså data som gjentar seg selv. Det tar opp mye plass og fører til resultater som er vanskelig å lese. Hvis man feks har en tabell hvor både kunde og varebestilling ligger i samme tabell vil man få dobbeltlagring på varene. Kunder kan bestille mange av de samme varene, eller en kunde kan bestille mange varer. Det gjør at man vil få opp samme kunde og samme vare unødvendig mange ganger. Derfor deler vi opp varer, bestillinger og kunder i forskjellige tabeller for å slippe dette. Redundans tar opp unødvendig plass, gir resultater som kan være vanskelig å lese og det blir vanskelig å oppdatere tabellen (feks hvis varer endrer pris).

Fan trap er en modelleringsfeil som gjør at man ikke får med viktige forbindelser. Disse oppstår når vi har M:1:1:M relasjon.

Fra et gitt institutt finner vi tilhørende campus, men derfra er det en forgreining (**vifte = fan**) ut til Ansatt.



Løsning:

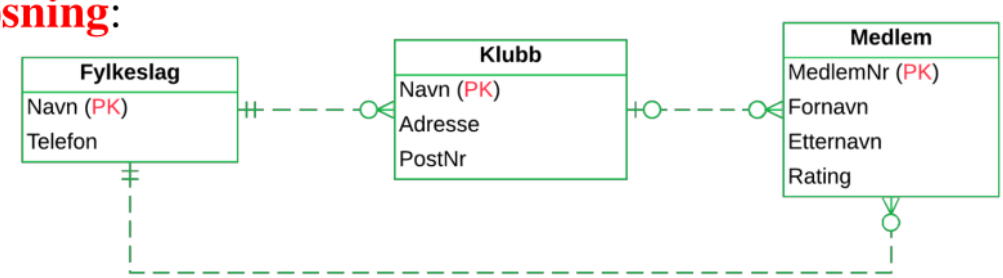


Chasm trap er når det er en relasjon mellom to entiteter som ved enkelte tilfeller ikke eksisterer, eller at relasjonen gjør at vi ikke får med alle tilfeller.

Mangelfull datamodell: Vi får ikke med hvilket fylkeslag klubbløse medlemmer tilhører.

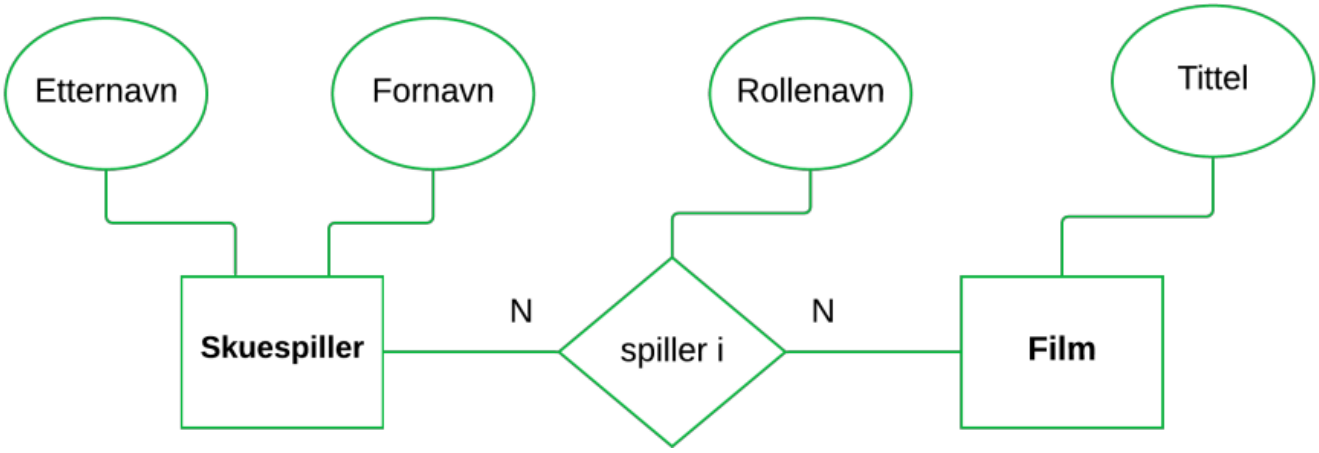
Modellen har «0 eller 1» inn mot Klubb fra Medlem, som gir ei «**kløft**» (**chasm**) mot Fylkeslag.

Løsning:



Dialekter

Ulike dialekter. UML, Chen-notasjon, MySQL Workbench, mm. Vi bruker ER (Entity Relation), også kalt kråkefot-notasjon. Chen-notasjon er ofte brukt, men mange tegneverktøy bruker kråkefot-notasjon fordi Chen-notasjon tegner attributtene i ovaler utenfor entitetene. Dette gjør at diagrammene krever mer plass. I Chen-notasjon tegner man entiteter som rektangler og forhold blir symbolisert med romber. Attributtene blir som nevnt tegnet utenfor entitetene i ovaler. Kardinaliteten blir vist med tekst, 1 og n. Identifiserende forhold blir angitt med doble streker på både romben og den svake entiteten som arver identifikator.



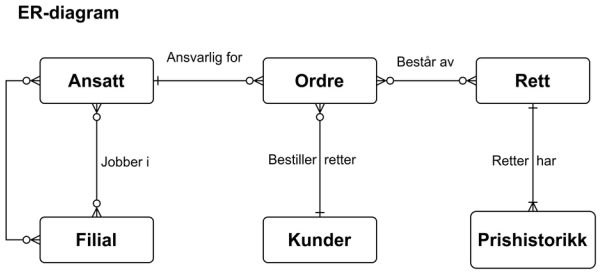
MySQL Workbench er heller ikke uvanlig. Den ser ganske lik ut som vanlig kråkefot-notasjon som vi har tegnet opp. Spesielt for Workbench er at den er tilpasset MySQL, bruker andre symboler på attributtene og viser hvilken datatype de har.

Dette kommer trolig ikke på eksamen.

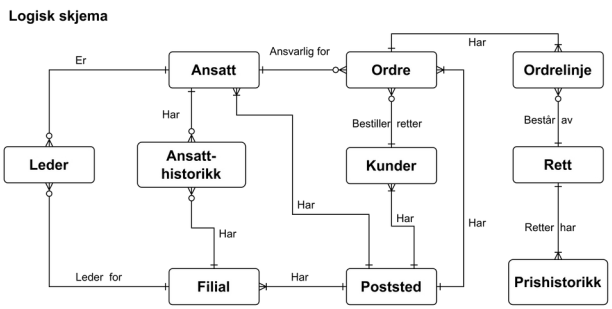
ER til logisk til attributter

Måten vi har modellert på er å lage tre modelleringsformer.

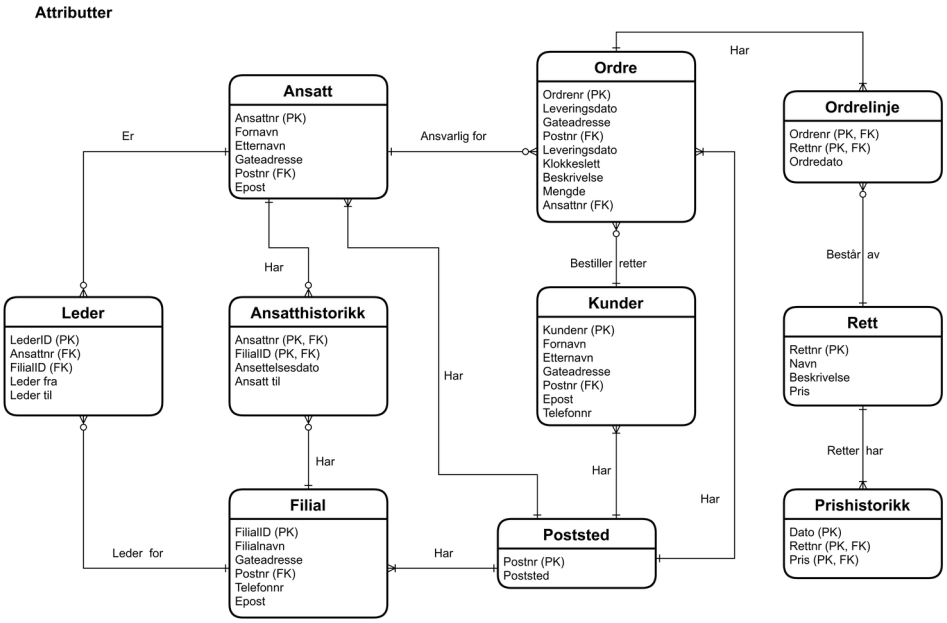
Vi begynner med et abstrahert ***ER-diagram*** med mulighet til mange-til-mange relasjoner. Vi tegner opp entitetene og relasjonene mellom de (med navn på relasjoner).



Så lager vi et ***logisk skjema***. Her bryter vi opp mange-til-mange relasjoner ved hjelp av hjelpetabeller og vi justerer også relasjonene. OBS: Vi kan ikke fjerne eller tilføye noe som ikke er i ER-diagram.



Til slutt legger vi inn ***attributter*** med nøkler i det logiske skjemaet.



Tips og triks

- Les oppgavetekst *nøye* og marker hvilke attributter og entiteter som trengs. Man får ofte oppgitt nøyaktig hvilke attributter som trengs.
- Hvis noe i teksten har attributter vil det alltid bli en egen entitet. Hvis noe i teksten ikke har attributter kan det ofte bare være en attributt i en annen entitet.
- Når man bryter opp mange-til-mange relasjoner i et ER-diagram vil alltid hjelpetabellen ha mange relasjon inn på hver side, og en-relasjon til de andre entitetene.
- Hjelpetabellen får alltid fremmednøkkel fra begge tabellene den er koblet til. Disse blir også ofte primærnøkkelen. NøkkelTabell1 (PK, FK), NøkkelTabell2 (PK, FK).
- En entitet som har mange-relasjon inn får alltid fremmednøkkel fra den andre entiteten
- Hvis man har en-til-en relasjon så kan man velge selv hvilken entitet som får fremmednøkkel. Det er ofte lurt å tenke på hvem som gir færrest nullmerker og gir mest mening.
- Man må alltid ta høyde for historikk! Altså at man lagrer data tilbake i tid. Det får innvirkning på relasjonene. Feks pris eller ansatte som endres over tid. Har man med historikk så vil man gjerne ha dato/tidspunkt som attributt.
- Det er sjeldent en-til-en relasjon når man tar høyde for historikk.
- For å få diagrammet på normalform er det viktig å ha atomære verdier. Dvs å dele opp i fornavn og etternavn. Det er da også lurt å lage en egen entitet for poststed.

- Det er nesten alltid null-mange relasjon og ikke en-mange. Fordi man må ta høyde for at det kan være tomme verdier ved registrering. En-mange blir kun brukt hvis det er et poeng at det skal være minst en.
- Skriv ALLTID navn på relasjonen
- Hvis det ikke er en naturlig identifikator (primærnøkkel) er det bare å lage et løpenummer. Feks AvdelingID. Hvis identifikatoren består av en kombinasjon av flere primærnøkler kan man alltids bytte den ut med løpenummer.
- Hvis man ikke klarer å finne på et fornuftig navn på en hjelpetabell kan man alltids kalle den navn1_navn2, el.
- I et logisk skjema kan man ikke innføre nye entiteter (med mindre det er hjelpetabell) fra ER-diagrammet.

Normalformer / normalisering

Redundans er at noe blir unødvendig gjentatt, altså overflødig informasjon. Dårlig tabelldesign gir redundans, noe som gir utfordringer når en database er i bruk. Vi får ***oppdateringsanomalier***. Når vi har dette blir det vanskelig å lage spørringer for å hente informasjonen man trenger. For å hindre redundans deler vi opp tabeller i enklere tabeller. Det er dette som er normalisering. Vi gjentar denne prosessen til all redundans er fjernet.

1NF – Første normalform

En tabell som er på første normalform inneholder kun *atomære verdier*. Dvs. at hver post i tabellen kun inneholder en verdi. Hadde vi hatt en kolonne for telefonnummer til personer og det i postene lå to telefonnumre ville dette bli brudd på første normalform.

2NF – Andre normalform

En tabell er på andre normalform dersom den

1. kun inneholder *atomære verdier* (på 1NF)
2. Den ikke inneholder *partielle avhengigheter*

Gitt tabell T(A,B,C,D) så vil B --> D være en *partiell avhengighet* dersom:

- B er en ekte delmengde av en primærnøkkel i T
- D er ikke med i primærnøkkelen i T.

Feks. Tabellen Ordrelinje(OrdreNr, VNr, Dato, Betegnelse) så vil VNr --> Betegnelse være en partiell avhengighet fordi VNr er en ekte delmengde av primærnøkkelen, Betegnelse er ikke med i primærnøkkelen og de er avhengige av hverandre. Derfor er eksempelet med Ordrelinje brudd på 2NF. For å få dette eksempelet over på 2NF må man bryte det opp i to tabeller:

Ordrelinje(OrdreNr, VNr*, Dato)
Vare(VNr, Betegnelse)

3NF – Tredje normalform

En tabell er på tredje normalform dersom den er 2NF (andre normalform) og den ikke inneholder *transitive avhengigheter*.
Gitt tabell T(A,B,C,D) så vil A --> C være en *transitiv avhengighet* i T dersom:

- A er primærnøkkel for T
- B eller C er ikke med i primærnøkkelen i T og B --> C

Feks: Ansatt(AnsNr, Fornavn, Etternavn, PostNr, Poststed)
PostNr --> Poststed, ingen av de er primærnøkkel og AnsNr er primærnøkkel, så vil det si at vi har en transitiv avhengighet. Da bryter denne tabellen med 3NF.

For å få denne på 3NF må vi igjen bryte opp tabellen:
Ansatt(AnsNr, Fornavn, Etternavn, PostNr*)
Poststed(PostNr, Poststed)

BCNF – Boyce-Codd NormalForm

En tabell er på Boyce-Codd NormalForm dersom enhver *determinant* er en *supernøkkel.* Eller enhver *minimal determinant* er en *kandidatnøkkel*.

En *determinant* er en kolonne som bestemmer en annen. Ved en funksjonell avhengighet A --> B så vil A være en *determinant*. Determinanter kan være sammensatt av flere kolonner (feks sammensatte primærnøkler). OBS: Selv om A --> A alltid gjelder (*triviell avhengighet)* så blir det ikke regnet som en determinant.

En *kandidatnøkkel* er en nøkkel som brukes til å identifisere en rad i en tabell. Den kan være primærnøkkel, men må ikke nødvendigvis være det.

En kolonne som bestemmer enhver annen kolonne kalles en *supernøkkel*. En *kandidatnøkkel* regnes som en *minimal supernøkkel*.

Tabeller på 3NF er ofte også på BCNF, men ikke alltid. Forskjellen er at 2NF og 3NF bygger på primærnøkler, mens BCNF bygger på determinant og kandidatnøkkel. For tabeller hvor kandidatnøkkelen er primærnøkkelen vil det derfor ikke være noen forskjell på BCNF og 3NF. Forskjellen dukker først opp i tabeller med overlappende kandidatnøkler.

Feks: Forestilling(Kinosal, Dato, KlSlett, FilmNr)
Kombinasjonen Kinosal, Dato og KlSlett bestemmer FilmNr og er primærnøkkel for tabellen. Denne kinoen blir en film alltid vist i den samme kinosalen. Da får vi en funksjonell avhengighet FilmNr --> Kinosal. Det betyr at tabellen får to overlappende kandidatnøkler:
Kinosal + Dato + KlSlett
FilmNr + Dato + KlSlett

Denne tabellen er på 3NF fordi den ikke inneholder partielle eller transitive avhengigheter. Men den er ikke på BCNF fordi hver determinant ikke er en supernøkkel.

BNCF: Forestilling(Dato, KlSlett, FilmNr)
Film(FilmNr, Kinosal)

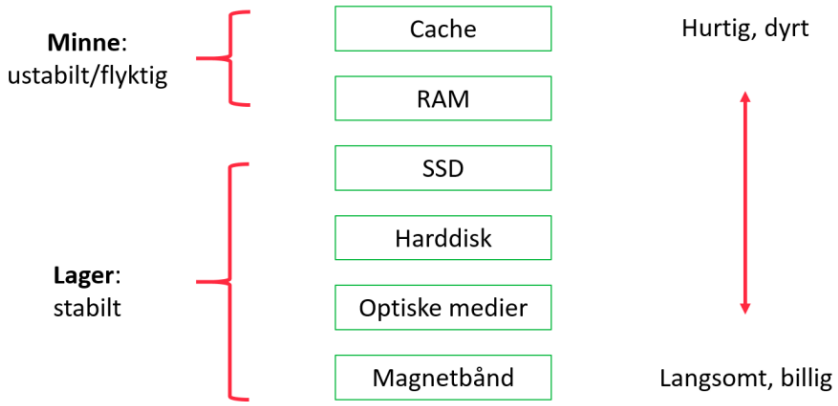
Her vil hver determinant være supernøkkel, og den er derfor på BCNF.

Denormalisering

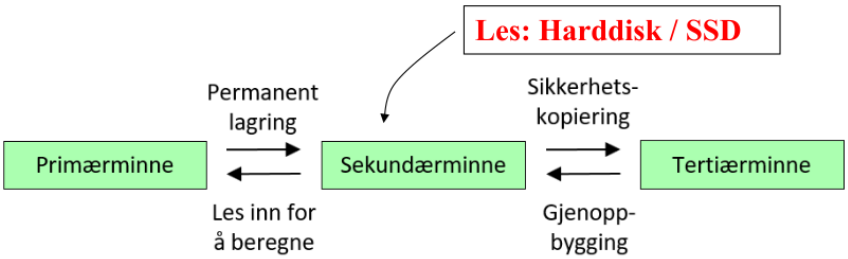
Noen ganger gjør vi det motsatte av normalisering, altså denormalisering. Normalisering gir en database uten redundans som gjør det enklere å utføre spørringer. Ulempen er at vi kan ende opp med et treigere databasesystem. Flere tabeller fører til økt behov for likekoblinger. Denormalisering innebærer at man godtar en viss mengde redundans for å oppnå et mer effektivt system. Vi sier at denormalisering fører til ***kontrollert redundans***.

Lagring

Lagringsmedier



Vi har ulike typer lagringsmedier og lagringsmåter. Vi har *minnet* (cache og RAM) som er den hurtigste formen for lagring. Det er som regel ikke stort nok til å holde hele databasen. Vi har *lager (disk)* som er mer stabilt, men langsomt. Data blir lest inn i minnet fra disk for å utføre operasjoner hurtig. Beregninger og oppdatere data som gjerne skal skje raskt. Data blir deretter skrevet tilbake til disk for mer stabilt (men treigere) lagring. Et DBHS flytter data kontinuerlig mellom minne og disk. Disk kan være SSD, harddisk, optiske medier og magnetbånd. Jo raskere et medium er desto dyrere og mer ustabilt er det. Lagringsmedier som er langsomme er ofte mer stabilt og billig. Derfor er det naturlig å utføre operasjoner i minnet som er raskere, for deretter å flytte det til minne/disk for lagring.



Lagring

Ethvert lagringsmedium kan sees på som en *nummerert sekvens av byte* nummerert fra 1 og oppover. Minnet er raskere enn disk og overfører data i blokker. Data på disken er derfor en *sekvens av blokker*, igjen nummerert fra 1 og oppover. En typisk blokkstørrelse er på 4kB = 4096 byter. En disk består i prinsippet av en endimensjonal tabell av celler som kan lagre 1 byte hver. De fleste verdier krever mer enn 1 byte, og vi må da finne en struktur for hvordan todimensjonale tabeller skal oversettes til endimensjonale i lagring. I en relasjonsdatabase er det vanlig å lage tabeller rad for rad, mens noen NoSQL databaser lagrer data kolonne for kolonne.

SQL	Rel.mod.	ER	Filer
tabell	relasjon	entitetstype	fil
rad	tuppel	entitetsforekomst	post
kolonne	attributt	attributt	felt

Filer

Fysisk blir databaser lagret i filer. En fil består av et antall poster, og hver post av et antall felt. Forenklet blir hver tabell lagret i en egen fil. For å få tak i en post må vi vite i hvilken blokk de ligger. Vi kan tenke at en tabell blir lagret rad for rad. En forekomst i en tabell vil da benytte flere poster etter hverandre. Vi må derfor tenke på hvor mange bytes vi trenger å sette av til de ulike postene i tabellen (filen) vår.

Representasjon av poster

Plassbehovet i en tabell avhenger av valg av datatyper. Vi har faste og variable lengder i datatyper. VARCHAR(n) er et eksempel på dette. Hvis ett eller flere felt har variabel type så får hele posten variabel lengde. Lagring av flere posttyper i samme fil kan også gi variabel lengde. INTEGER er eksempel på en datatype med fast lengde (4 byter). Ved variable lengder blir det ofte opprettet tomme plasser for å «holde av» plass i minnet. Hvis vi har en tabell bestående bare av kolonner med fast lengde vil vi få en fastsatt størrelse på filen. Det gjør det effektivt og raskt å finne en post. I en tabell med variabel lengde vil det være vanskeligere å finne poster. Vi kan bruke skilletegn, men gjenfinning krever i så fall at vi søker i tabellen.

Blokker

En harddisk er mye treigere enn minnet. Når vi leser fra/skriver til harddisken tar vi en større mengde av gangen. Dette kalles blokker. En *blokk* er den minste enheten for overføring av data mellom minnet og lagring. En typisk blokkstørrelse er på 4kB (4096 bytes). En blokk inneholder da typisk mange poster (rader i tabellen). DBHS holder oversikten over hvilken data som ligger i hvilke blokker. En fil blir gjerne lagret i et antall blokker. Det settes av litt ledig plass i hver blokk, noe som gjør det lettere å sette inn/slette poster. *Fyllingsgraden* (feks 80%) kan styres i mange DBHS.

Fil delt inn i blokker

	KNr	Fornavn	Etternavn	Telefon	
Post 1	1	Elias	Hansen	99 88 77 66	Blokk 1
Post 2	2	Hulda	Akselsen	31 45 88 21	
Post 3	5	John	Boine	23 94 53 18	
	...	...	...	...	
Post 58	63	Mari	Gygre	55 66 77 88	
Post 59	64	Michael	Svensen	19 82 37 64	Blokk 2
Post 60	69	Robert	Romman	91 28 73 46	
Post 61	72	Laura	Eika	64 37 82 19	
	...	...	...	...	
Post 116	129	Andreas	Karlsen	51 15 83 38	
	...	...	...	...	

Filstrukturer – Sorterte og ikke-sorterte filer

Postene i en fil må ordnes på en eller annen måte. Den letteste måten er å organisere poster i den rekkefølgen de ble registrert. Fordi registreringsrekkefølgen ikke nødvendigvis har noen sammenheng med innholdet i tabellen får vi poster i tilfeldig rekkefølge. Dette kalles Ikke-sorterte filer eller heap-filer. Fordelen med disse er at innsetting av poster er veldig effektivt, fordi vi bare legger til på slutten. Ulempen er da at å finne de igjen tar lang tid. Vi må lese filen fra start til slutt med et *sekvensielt søk*. I gjennomsnitt må vi søke gjennom halve filen for å finne en post, og i verste fall hele filen. F.eks: En fil med 10 000 poster bestående av 200 blokker. Et sekvensielt søk vil da kreve 200/2 = 100 diskaksesser i snitt, eller i verste fall 200.

Den andre løsningen er å sortere postene etter en kolonne, feks primærnøkkelen i tabellen. Dette gir en sortert fil, også kalt *sekvensiell fil*. Denne måten å sortere på gjør det mulig å bruke effektive søketeknikker. Et eksempel på dette er et *binært søk*. Da slår man opp midten i fila, så ser man om det man leter etter ligger før eller etter. Så deler man igjen og fortsetter til man har funnet posten. Hvis vi tar utgangspunkt i 10 000 poster med 200 blokker vil man med binærsøk kunne garantere at man finner den innen 8 oppslag.

Forskjellen blir selvfølgelig betydelig større etter hvor stor fila er.

Hashing

Hashing er en teknikk for å direkte søke etter ønsket data på en disk, uten å kjenne til indeks strukturen. Hashing organiserer postene i en fil ved hjelp av en hash-funksjon, en matematisk funksjon som beregner en blokkadresse basert på verdien til et av feltene i filen. Dette feltet kalles hash-nøkkelen. Hash-nøkkelen som genereres er mindre enn den originale verdien og gjør det lettere og raskere å søke etter verdien vi ønsker. Ved å bruke hash sparer man plass, og det er også nyttig hvis man feks skal lage passord som vi ikke ønsker å ha i klartekst, da kan vi kryptere de ved hjelp av en hash. Når man hasher flytter man også data i blokker på passe størrelse, i motsetning til ulik data som lagres i en blokk. I utgangspunktet kan man med et hash-søk bruke kun 1 diskaksess til å finne det man leter etter.

Fra powerpoint:

En hashfunksjon avbilder verdier på blokk-adresser.

– Setter av n antall blokker til filen.

– Leke-eksempel: h(x) = x MOD 17 (rest ved divisjon)

• Velger et felt som hash-nøkkel.

– Lagring av post: Anvend h på hash-nøkkelen.

– Gjenfinning: Anvend h på hash-nøkkelen.

• Hva om h sender for mange poster til samme blokk?

– Bruk ”overflyt-blokker” (men dette medfører litt søking).

– Ønsker å minimalisere både plassforbruk og antall kollisjoner.

Indeksering

En indeks kan sees på som et stikkordregister i en bok. Det er en datastruktur som kan effektivisere søk. Den består av *søkenøkler* og *pekere* (søkeord og referanser). I en bok er søkenøklene litt som nøkkelord som forekommer i boka og pekere er sidetall. En indeks er alltid sortert med hensyn på søkenøkkelen.

Nøkkel	Adresse
Akselsen	
Boine	
Eika	
Gygre	
Hansen	
Karlsen	
Romman	
Svensen	

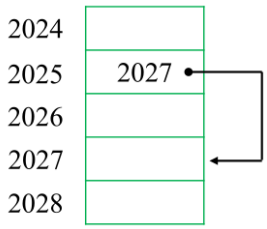
KNr	Fornavn	Etternavn	Telefon
1	Elias	Hansen	99 88 77 66
2	Hulda	Akselsen	31 45 88 21
5	John	Boine	23 94 53 18
63	Mari	Gygre	55 66 77 88
64	Michael	Svensen	19 82 37 64
69	Robert	Romman	91 28 73 46
72	Laura	Eika	64 37 82 19
129	Andreas	Karlsen	51 15 83 38

Illustrasjonen viser en kundefil med indeks på etternavn, der pekerne er adresser (et tall) til rader i tabellen.

Indekser effektiviserer søk fordi indeksen er mindre og den er sortert. Til gjengjeld tar det noe mer lagringsplass. Litt som at et stikkordsregister i en bok tar opp noen sider ekstra. Man får også merarbeid ved oppdateringer ved at både datafilen og indeksen må oppdateres. Listen i en indeks vil være sortert med hensyn på søkeord og databaseindekser vil alltid være diskadresser.

Pekere (referanser)

Et lagringsmedium er i prinsippet en lang byte-sekvens. Enhver celle har en adresse (et tall). Hvis en celle inneholder adressen til en annen sier vi at den er en peker (referanse). Den peker/referer til den andre. Vi tegner ofte pekere som piler.



- Celle på adresse 2025 inneholder adressen (tallet) 2027.
- Kan symboliseres ved en pil fra celle 2025 til celle 2027.

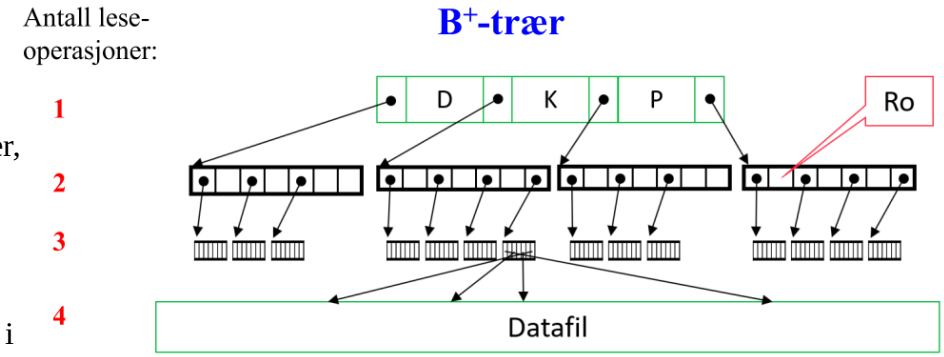
Tette og ikke-tette indekseres

En tett indeks (dense indices) har en peker for hver eneste post i filen. En ikke-tett indeks har bare en peker til hver eneste blokk i filen. Fordi en blokk inneholder flere poster vil en ikke-tett indeks være mindre enn en tett indeks. Det kan maks være 1 ikke-tett indeks per datafil, fordi en datafil kun kan være sortert på en måte, feks kundenr. Bare indeksen på kundenr. Kan da være ikke-tett.

B^+-trær

Indekser, særlig tette, blir ofte så store at man oppretter indekser til indekser. Indeksene får da en tre-struktur. Dette kaller vi flernivå-indekser. Indeksene på de lavere nivåene kan være ikke-tette fordi en indeks alltid vil være sortert. Søking i flernivå-indekser skjer fra roten og utover mot grenene. Antall diskaksesser vi trenger avhenger av dybden på treet.

B^+-trær (balansert tre) er en vanlig form for flernivå-indeksering. Det er like mange diskaksesser for alle søkeverdier, og treet er derfor balansert.



F.eks: Vi skal finne post der etternavn = 'Roheim'. Les inn rot-blokken (øverste nivå). Søk i rot-blokken (her alfabetet) og gå til R, helt til høyre. Så gjentar vi denne typen søk hele veien til vi kommer til riktig blokk i datafilen (der all dataen ligger, altå det indeksene peker på).

En ulempe med indekser er at innsetting og sletting i den underliggende datafilen medfører at også indeksene må bli oppdatert. Og denne jobben blir spesielt krevende i balanserte indekser. Oppdateringen kan føre til at man må splitte nodene i treet og det kan føre til at vi må legge til et nivå eller slå sammen. På tross av dette er B^+-trær en effektiv teknikk og blir brukt som standard indekseringsteknikk i mange systemer.

Valg av datatyper

Når vi lager tabeller i et databasesystem er det viktig å tenke over hvilke datatyper vi velger. Vi ønsker å ha en fleksibel nok tabell, samtidig som vi ønsker at den skal være så effektiv som mulig. Det er spesielt viktig i store databasesystemer!

Tekst:
Vi bruker CHAR(n) hvis verdiene har en fast lengde. Feks fødselsnummer som alltid er på 11 tegn. Hvis vi har en varierende lengde bruker vi VARCHAR(n). Det er viktig å sette lang nok maks-grense. Hvis vi har veldig lange tekster (over 65535 tegn) så må vi bruke andre datatyper som MEDIUMTEXT eller LONGTEXT.

Tall:
Som med andre datatyper ønsker vi å velge så liten datatype som mulig, men slik at alle mulige verdier får plass. Vi skiller også mellom heltall og desimaltall. På sistnevnte kan vi også spesifisere om det er et fast antall desimaler. TINYINT(n) gir heltall mellom -128 til 127, INT(n) gir et høyt heltall. Med desimaltall bruker vi DECIMAL(n,d) eller DOUBLE(n,d)

Tid:
Dato, klokkeslett, år? Formatet til DATE er YYYY-MM-DD

Tekst eller tall? Trenger du å regne med verdien? Hvis ikke kan man velge å lagre de som noe annet enn en tallverdi.

Valg av indekser

Kolonner som med fordel kan indekseres:

- Kolonner man ofte søker i eller sorterer på
- Primærnøkler (som regel automatisk indeksert)
- Fremmednøkler (kan være automatisk indeksert)
- Kolonner som ofte brukes som koblingsnøkler i spørringer

Kolonner som ikke bør/kan indekseres:

- Kolonner som kun inneholder mange like verdier (feks ja/nei eller 0/1)
- Bilder, video, lydklipp og annen media
- Lange dokumenter (finnes spesielle teknikker, brukes av feks søkemotorer)

Det er viktig å tenke på om indeksering er lønnsomt i systemet. Indekser tar opp plass og de må vedlikeholdes for hver oppdatering. Det er derfor ikke *alltid* lønnsomt å indeksere. Man bør også tenke på bruksmønster og hva brukere ofte søker etter. Det er vanlig å fjerne/endre indekser etter at databasen er tatt i bruk, for da ser man lettere hva bruksmønsteret er.

Transaksjoner

En transaksjon er en logisk operasjon mot en database og består av en eller flere SQL-spørringer og kan bestå av en eller flere tabeller. Kommandoene COMMIT og ROLLBACK bruker for å bekrefte eller angre en transaksjon.

En database lagres på disk, men må leses inn i hurtigminne for å gjøre operasjoner mot den. Fordi ting kan gå galt, eller flere jobber mot samme database kan det skje ting mellom hurtigminne og lagring, og det er derfor viktig at DBHS kan håndtere feilsituasjoner. Og ikke minst håndtere at det arbeides mot databasen.

En operasjon er enkelt forklart når man gjør noe med en database. Dette gjøres ofte med SQL-spørringer. For eksempel å sette inn en ny rad med verdier i en tabell. En sammensatt operasjon er når vi foretar mange handlinger på en gang. For eksempel en ordrebestilling vil føre til innsetting av data i flere tabeller, slik som ordrelinje, ordre og vare. Da får vi en sammensatt operasjon, hvor mange ting skal skje på en gang.

❑ Ansatt 22 mottar bestilling av 5 enheter av produkt 4034 fra kunde 1003 – og oppretter ordre 2020:

```
START TRANSACTION;
INSERT INTO Ordre(OrdreNr,KNr,AnsNr)
VALUES (2020,1003,22);
INSERT INTO Ordrelinje(OrdreNr,VNr,Antall)
VALUES (2020,'4034',5);
UPDATE Vare
SET Antall = Antall-5
WHERE VareNr='4034';
COMMIT;
```

❑ Etter den første/de to første innsettingene (INSERT) er ikke databasen i en lovlig tilstand.

Transaksjoner i MySQL

❑ **Auto-commit** er standard. Kan skrus av:

```
SET autocommit = 0;
```

❑ Eller start **eksplisitt** med START TRANSACTION:

```
START TRANSACTION
INSERT INTO Ordre(OrdreNr,KNr,AnsNr)
VALUES (2020,1003,22);

-- Flere kommandoer …
COMMIT;
```

Feilsituasjoner

Sett at det oppstår et strømmbrudd like etter at en INSERT-setning er utført, men at databasen ikke blir oppdater. Da vil ikke nødvendigvis tabellene stemme overens. Når systemet starter opp må halvferdige operasjoner identifiseres og håndteres. DBHS må sikre at enhver transaksjon blir utført i sin helhet, eller ikke i det hele tatt dersom det oppstår en feil.

De fleste databasesystemer har flere brukere. Når mange brukere er logget inn i systemet samtidig, kan transaksjonene deres ødelegge for hverandre. Et DBHS må ha mekanismer for å håndtere slike konflikter. I et system for å bestille konsertbilletter vil man først sjekke om det er ledige billetter før man kjøper de. Hvis to personer håndterer en slik operasjon samtidig vil det oppstå en konflikt, og det må man hindre at skjer. Vi kan ikke nødvendigvis styre at to brukere utøfrer operasjoner mot databasen, så et DBHS må ha mekanismer for å sikre at transaksjonene blir håndtert korrekt.

<u>Feilsituasjoner</u>

Databasesystemer er i kontinuerlig drift over lang tid, og man må regne med at feilsituasjoner oppstår. *Strømbrudd* og *diskkrasj* er to eksempler på vanlige feil som kan oppstå. Systemet må derfor kunne loggføre transaksjoner etter hvert som de blir utført for å kunne fikse opp etter alvorlige feil.

Samtidige brukere

En database har som regel mange brukere og flere kan være pålogga og aktive samtidig. DBHS må sørge for at ikke to brukere starter transaksjoner som oppdaterer samme data til samme tid. Man må ha en koordineringsløsning.

Håndtering av feilsituasjoner

Vi har COMMIT og ROLLBACK for å jobbe med transaksjoner. Hver enkelt kommando blir betraktet som en transaksjon. Det kalles auto-commit. Alle kommandoer frem til COMMIT eller ROLLBACK tilhører samme transaksjon. Enkelt forklart er COMMIT bekreft og ROLLBACK angre. Selv om vi har utført SQL-spørringer blir ikke effekten endelig før vi har brukt COMMIT. ROLLBACK gjør at transaksjoner ikke får noen effekt og transaksjonen blir angret eller «rullet tilbake».

Kommandoen ROLLBACK brukes til å angre en halvveis utført transaksjon. Feil inndata kan føre til at det ikke gir mening å fullføre transaksjonen. Det blir en enkel måte å håndtere feilsituasjoner på. Transaksjoner kan også bli avbrutt på andre måter. Brukerfeil kan feks skje ved at noen sletter en tabell ved et uhell. Feilprogrammering av en transaksjon kan føre til avbrudd. Manglende kontroll med inndata kan gi opphav til meningsløse beregninger, som å dele på 0. Fordi minner er avhengig av strøm skjer permanent lagring av data på disk. Transaksjoner fører til at data hele tiden går mellom minne og disk. Databasen kan få problemer fordi en disk er full eller det skjer en strømstans og dataen eksisterer kun i minnet og derfor gå tapt. Det er derfor viktig å kunne gjennoppbygge databasen etter vi har fått feil.

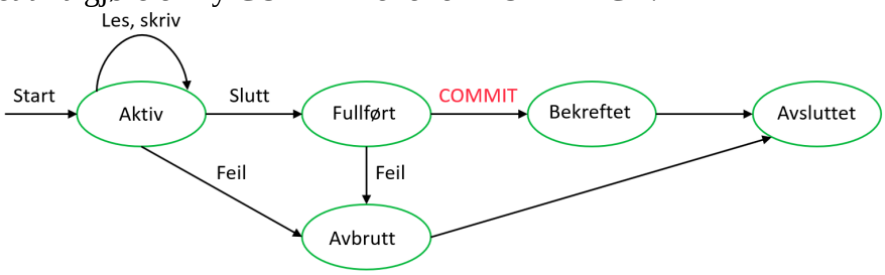
Transaksjonslogg

En transaksjonslogg er en fil på disken som inneholder data om alle transaksjoner som er utført mot databasen. En egen modul i DBHS har som oppgave å vedlikeholde loggen. Det blir registrert at en transaksjon starter, at den oppdaterer elementer og at den utfører COMMIT og ROLLBACK. Fordi loggen er på disk vil den overleve at databasen går ned og fordi DBHS registrerer operasjoner i loggen før den skriver til databasen, blir det mulig å gjenopprette ved feil.

TransID	Table	RowID	Attribute	Before	After
101	*** BEGINTRANS				
101	VARE	102375	ANTALL	112	113
101	VARE	102542	ANTALL	56	66
101	*** COMMIT				

For å skille transaksjoner fra hverandre får de tildelt et nummer i loggen. På bildet er det en transaksjonslogg for transaksjon 101. Hvis antallet viser seg å være feil kan vi nå enkelt se verdiene før og etter transaksjonen og det blir mulig å endre feil.

Hvis en transaksjon ikke går gjennom får vi også dette opp i loggen som ikke-fullførte transaksjoner. Da har vi mulighet til å gjøre en ny COMMIT eller en ROLLBACK.



Generelt blir flere transaksjoner utført samtidig, derfor vil ikke oppføringene nødvendigvis stå samlet i loggen. Når samtlige operasjoner er loggført sier vi at transaksjonen går fra en *aktiv tilstand* til en *fullført tilstand*. Systemet vil da gjennomføre kontroller og hvis alt er i orden skrives det COMMIT i loggen og den går over til en *bekreftet tilstand.* En transaksjon kan bli avbrutt på ethvert tidspunkt før den er i en bekreftet tilstand. Selv om en transaksjon er bekreftet er det ikke sikkert at alle oppdateringene er skrevet til databasen, men fordi den er bekreftet er systemet forpliktet til å gjenomføre transaksjonen. Når en database går ned vil innholdet i transaksjonsloggen brukes for å gjenopprette tilstanden til så nær mulig til feiltidspunktet som mulig.

Gjennoppretting ved hjelp av transaksjonslogg

En instansefeil ødelegger ikke data på disk, og vi kan da starte opp databasen igjen ved hjelp av transaksjonsloggen for å få innholdet tilbake. Ved diskkræsj er ikke det tilstrekkelig, og man må da ha en sikkerhetskopi for å gjennoppbygge databasen.

Vi kan bruke transaksjonsloggen på to måter ved gjenoppretning (recovery):

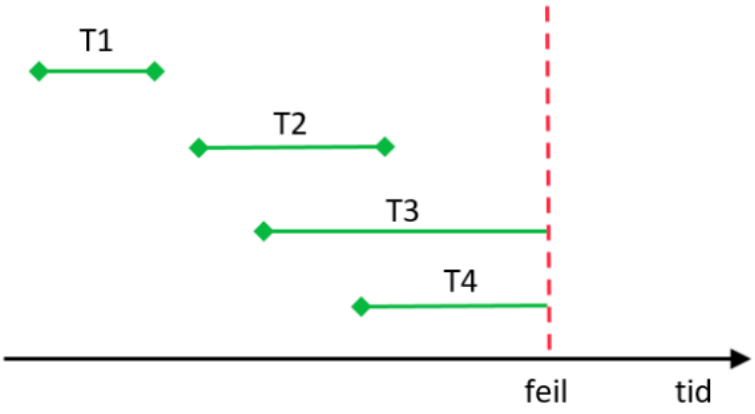
UNDO – Fordi loggen inneholder både gamle og nye verdier for alle skriveoperasjoner er det mulig å reversere effekten av en transaksjon ved å lese loggen baklengs og erstatte nye verdier med gamle. Vi sier at transaksjonen ble rullet tilbake.

REDO – Ved å lese loggen forlengs kan avbrutte transaksjoner gjennomføres på nytt. Da vil gamle verdier bli overskrevet av nye.

Forskjellige DBHS har ulike teknikker for gjennoppretning:

Umiddelbar oppdatering – Databasen blir oppdatert underveis i en transaksjon selv om man alltid sørger for at skrive-operasjoner blir loggført før de får effekt på databasen. For å gjøre ROLLBACK må effekten transaksjonen har hatt må angres. Det gjøres ved UNDO.

Utsatt oppdatering – Oppdateringen av databasen blir utsatt til alle operasjoner er registrert i loggen. For å gjøre ROLLBACK trenger man bare notere ROLLBACK i transaksjonsloggen fordi databasen enda ikke er oppdatert.

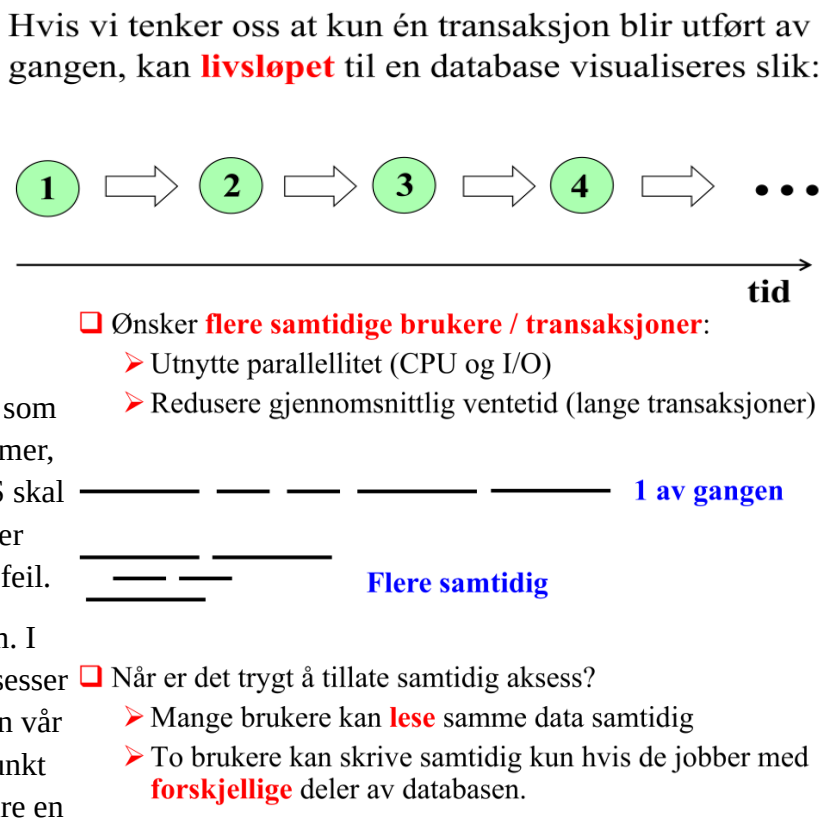


Figuren til høyre viser fire transaksjoner i et hendelsesforløp der det oppstår instansfeil. T1 og T2 har skrevet COMMIT til loggen og ble bekreftet før feilen inntraff. T3 og T4 var under utførelse. Kanskje hadde T3 rukket å gjennomføre flere skrive-operasjoner, mens T4 kun hadde en leseoperasjon. Med umiddelbar oppdatering må effekten av T3 rulles tilbake. Med utsatt oppdatering er ikke det nødvendig. Deretter må T3 og T4 bli startet på nytt. T1 og T2 er bekreftet, men DBHS kan ikke nødvendigvis ikke vite om transaksjonene har blitt oppdatert i databasen og må i så fall kjøre T1 og T2 på nytt basert på registreringer i loggen (REDO).

I teorien kan det bety at DBHS må gjøre igjen samtlige transaksjoner. For å gjøre gjennoppbygging mer effektivt vil DBHS med jevne mellomrom skrive *sjekkpunkter* til loggen. En enkel måte er å fullføre samtlige transaksjoner uten at nye blir startet, sørge for at alle operasjoner er loggført også notere sjekkpunktet i loggen. Ved gjenoppbygging er det tilstrekkelig å lete bakover i loggen til siste sjekkpunkt.

Flerbruksdatabaser

En *tilstand* er innholdet i databasen på et bestemt tidspunkt. En transaksjon bringer databasen fra en tilstand til en annen. Til venstre er en illustrasjon av hvordan det vil se ut om vi bare har en transaksjon som utføres av gangen.



Det er ofte ikke tilfelle. Vi har vanligvis databaser hvor det er flere samtidige bruker og flere transaksjoner som skjer samtidig. Dette kan føre til problemer, men gjør prosessen mer effektiv. DBHS skal sørge for at flere kan utføre transaksjoner mot en database uten at det skal oppstå feil.

En *prosess* er en utførelse av et program. I en datamaskin har vi gjerne mange prosesser som skjer samtidig. Vi deler gjerne tiden vår mellom ulike prosesser. På et gitt tidspunkt er det mange prosesser «i live», men bare en er aktiv. *Tidsdeling* vil si at maskinen bruker litt tid på hver prosess. Først utføres noen instruksjoner fra prosess A, deretter fra B, så noen fra C, så tilbake til A, osv. For en enkelt prosess har dette samme effekt som at den ble utført alene. Det samme prinsippet gjelder for databaser.

ACID-egenskapene

En database bør ha følgende egenskaper knyttet til transaksjoner:

Atomicity (Alt eller ingenting) – Enten blir en transaksjon utført i sin helhet, eller ikke i det hele tatt.

Consistency (Konsistens) – Hvis databasen er i en konsistent tilstand før en transaksjon starter, skal databasen også være konsistent når transaksjonen er ferdig utført. Fra en lovlig tilstand til en annen lovlig tilstand.

Isolation (Ingen innblanding) – Selv om flere transaksjoner blir utført samtidig, skal de ikke ødelegge for hverandre. Effekten av transaksjoner skal ikke kunne observeres av andre transaksjoner.

Durability (Varighet) – Når en transaksjon er bekreftet, så skal effekten av transaksjonen bli permanent lagret. Dette skal gjelde selv om det oppstår feil på et seinere tidspunkt. Commitede transaksjoner skal lagres og ikke gå tapt pga feil.

Disse egenskapene blir kalt ACID-egenskapene

Les – Beregn – Skriv

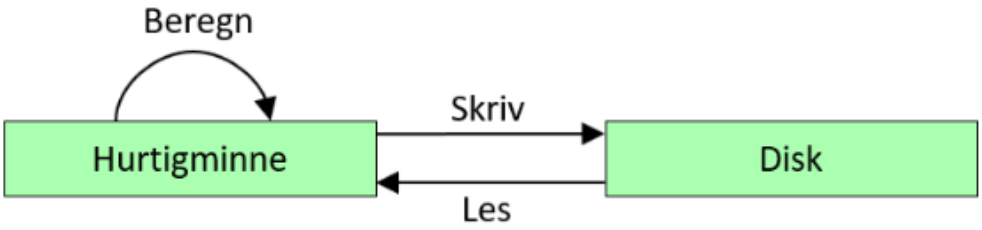
Ta et eksempel for håndtering av en transaksjon for varebestilling:

1. Les antall varer på lager (27) inn fra disken og lagre det på en bestemt plass i minnet

2. Beregn (27-3) og oppdater den avsatte plassen i minnet.

3. Skriv ny verdi (24) tilbake til disken for permanent lagring.

Les-Beregn-Skriv er ikke en *atomær operasjon* men sammensatt av flere enkelte operasjoner. Illustrasjonen viser hvordan dette fungerer. Det kan oppstå problemer når flere brukere oppdaterer data på samme tid.



Utfordringer med samtidige brukere

Når flere transaksjoner jobber mot samme dataområde på samme tid kan det oppstå konflikter. Noen eksempler:

<u>Tapt oppdatering (lost update)</u>

Lokal kopi T1	T1	Verdi A på disk	T2	Lokal kopi T2
120	Les inn	120		
110	Tell ned med 10	120	Les inn	120
110	Skriv til disk	110	Øk med 20	140
		140	Skriv til disk	140

tid

T1 og T2 skal begge oppdatere samme verdi A som er lagret på disk. T1 skal telle ned verdien med 10, mens T2 skal øke verdien med 20. Hvis A har en startverdi på 120, skal samlet effekt av T1 og T2 bli at A økes til 130 (120-10+20). Hvis de to transaksjonene blir startet på likt, kan imdlertid situasjonen på illustrasjonen skje. Dette kalles en *tapt oppdatering*. Det skyldes at oppdatering av verdier på disk er en sammensatt les-beregn-skriv operasjon. T1 leser inn verdien 120 fra disken til minnet og oppdaterer denne verdien. Oppdateringen har ikke effekt på verdien lagret på disken. Når T2 også leser inn fra disk, har den ikke fått med seg oppdateringen til T1. Deretter skriver T1 sin oppdaterte verdi 110 til disken. T2 oppdaterer samtidig sin lokale verdi (fra 120 til 140) og det hele avsluttes med at T2 skriver 140 til disken. Oppdateringen til T1 går dermed tapt.

Inkonsistent analyse (incorrect summary)

Tapt oppdatering involverer to eller flere transaksjoner som alle endrer databasen. Det kan oppstå feil også med to transaksjoner der den ene kun leser av databasen.

La oss tenke oss at det foregår vareopptelling samtidig med mottak av en leveranse. Man kan da risikere at opptellingen tar med den nye leveransen for noen varer, men ikke alle. Resultatet vil da verken samsvare med faktisk innhold før levering, eller faktisk innhold etter levering. Transaksjonen som utfører opptelling har observert et mellomresultat av transaksjonen som håndterer leveranser, og det bryter med prinsippet om ingen innblanding. Denne situasjonen kalles *inkonsistent analyse.*

T1	A	B	T2	Lokal sum
	10	50	Nullstill sum	0
	10	50	Legg til A i sum	10
Øk A med 10	20	50		10
Øk B med 20	20	70		10
			Legg til B i sum	80

tid

T1 skal øke A med 10 og B med 20. T2 skal legge sammen A og B, og kanskje skrive ut resultatet. Ved start er summen av A og B lik 60. Etter at T1 er utført er summen 90. T2 får imidlertid summen til å bli 80. Feilen er at T2 bruker gammel verdi av A, men ny verdi av B.

Angret oppdatering (dirty read)

Transaksjoner kan angres med ROLLBACK. Tilbakerulling av en transaksjon kan imidlertid skape problemer dersom den har skrevet til databasen og disse verdiene er lest av andre transaksjoner. Denne situasjonen kalles *angret oppdatering*.

T1	A	T2
	120	
Tell ned A med 10	110	
	110	Les inn A og bruk denne verdien
ROLLBACK	120	

tid

T1 oppdaterer A, men blir seinere angret. T2 avleser den midlertidige effekten T1 har på A og bruker denne verdien videre, kanskje til å oppdatere en annen del av databasen. Når T1 blir angret endres A tilbake til opprinnelig verdi. T2 jobber dermed på en angret verdi. Igjen er problemet at mellomresultat blir avlest av en annen transaksjon.

Låsemekanismer

Samtidighetsproblemer skyldes at to eller flere transaksjoner operer på samme data til samme tid. Den opplagte løsningen er å innføre en køordning som forhindrer samtidig bruk. Den aller enkleste løsningen er å utføre transaksjoner en etter en. Dette kalles ***sekvensiell utførelse***. Det vil åpenbart hindre alle samtidighetsproblemer, men er veldig restriktivt. To transaksjoner som leser eller skriver til forskjellige deler av databasen kan aldri komme i konflikt, selv om de blir utført samtidig. Datamaskiner kan utføre flere oppgaver samtidig. Sekvensiell utførelse av transaksjoner utnytter ikke denne egenskapen og brukerne kan oppleve lange ventetider. Blant annet er lange transaksjoner som stenger ute korte transaksjoner problematisk.

En *lås* er en mekanisme som regulerer tilgangen til deler av en database. En transaksjon som trenger tilgang til et bestemt område av databasen vil be DBHS om en lås på dette området. Hvis det ikke blir innvilget vil DBHS kunne stenge andre transaksjoner ute fra området. Slike transaksjoner blir satt i

ventekø. DBHS må holde styr på mange ventekøer, og det er transaksjonen som må be om låser. En transaksjon kan låse større eller mindre deler. Fra hele databasen, en tabell, en rad eller en celle.

Vi skiller mellom *leselåser* (exclusive locks) og *skrivelåser* (shared locks). Samtidighetsproblemer involverer ofte en eller flere transaksjoner som oppdaterer databasen. Slike transaksjoner kaller vi *skrivere*. Transaksjoner som kun avleser innholdet i databasen kaller vi *lesere*. Det vil aldri oppstå samtidighetsproblemer med kun lesere. Dette er grunnen til at DBHS skiller mellom lese- og skrivelåser. En leser som trenger tilgang til element X må sette en leselås på X. På den måten vil eventuelle skrivere ikke få tilgang mens man leser av data. Derimot kan vi slippe inn andre lesere (som også kan sette leselås). En transaksjon som vil oppdatere element X må sette en skrivelås på X. Det vil stenge alle andre transaksjoner ute, både lese og skrive. Transaksjoner som ikke får innvilget skrivelås fordi en annen transaksjon allerede har satt en lås blir satt på vent. DBHS må holde rede på hvilke transaksjoner som venter på å få innvilget forespørsel om lås på de ulike områdene. Når en transaksjon opphever en lås vil DBHS sjekke om det finnes transaksjoner i ventekø for dataområdet og i så fall vekke opp en slik transaksjon.

Lokal kopi	T1	Verdi A på disk	T2	Lokal kopi
	Skrivelås på A	120		
120	Les inn	120	Skrivelås på A?	
110	Tell ned med 10	120	**Vent**	
110	Skriv til disk	110	**Vent**	
	Lås opp A	110	**Vent**	
		110	Les inn	110
		110	Øk med 20	130
		130	Skriv til disk	130
		130	Lås opp A	

tid

Illustrasjonen viser bruk av låser for å forhindre tapt oppdatering.

Serialiserbare forløp

Låser løser ikke alle samtidighetsproblemer. Grunnen til det er at transaksjoner kan være sammenfletta av flere enklere operasjoner. Selv om låser regulerer tilgangen til hvert enkelt dataområde kan det oppstå problemer ved at to transaksjoner opererer på flere felles områder i forskjellig rekkefølge.

For å utføre transaksjoner må vi utføre alle deloperasjonene i alle transaksjonene i en eller annen rekkefølge. En bestemt rekkefølge kaller vi et *forløp* (schedule). Hvis vi velger å utføre transaksjoner en etter en har vi et ***sekvensielt forløp***. Slike forløp er alltid korrekte men ineffektive. For å få bedre effektivitet ønsker vi at så mye som mulig kan gjøres samtidig, men på en måte som at transaksjonene blir utført sekvensielt og korrekt. Et ***serialiserbart forløp*** er et forløp som har samme effekt som et sekvensielt forløp. Transaksjonene blir flettet sammen. Oppdatering av en verdi på disk skjer vanligvis ved tre operasjoner: Innlesing, oppdatering i minnet og tilbakeskriving til disk. For å forenkle forløpene slår vi de tre operasjonene sammen. Det blir innført betingede operasjoner på formen «hvis betingelsen er sann så aksjon».

Transaksjon T1 skal telle ned både A og B med 10. Transaksjon T2 skal øke A og B med 10, men bare dersom de i utgangspunktet har positive verdier. Hvis vi lar både A og B ha startverdi 10, vil et forløp med T1 først og deretter T2 gi sluttverdi 0 for begge lokasjoner (T2 har ikke effekt). Kjører

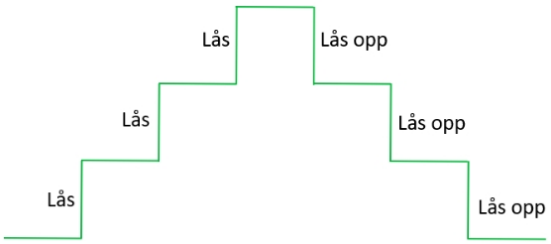
T1	A	B	T2
Skrivelås A	10	10	Skrivelås B
Tell ned A med 10	0	20	Øk B med 10 hvis større enn 0
Lås opp A	0	20	Lås opp B
Skrivelås B	0	20	Skrivelås A
Tell ned B med 10	0	10	Øk A med 10 hvis større enn 0
Lås opp B	0	10	Lås opp A

tid

NB! Ikke alle forløp er serialiserbare selv om de bruker låser.

vi T2 først, deretter T1, vil både A og B få uendret sluttverdi på 10. Når de to transaksjonene blir utført samtidig oppstår det problemer, selv om låser sikrer at de ikke modifiserer samme verdi til samme tid. A får sluttverdi 0 og B sluttverdi 10, altså ikke en effekt som samsvarer med noe sekvensielt forløp. T1 oppdaterer A først og deretter B, mens T2 oppdaterer verdiene i motsatt rekkefølge. Den første oppdateringen i T1 gjør at A ikke er positiv, og T2 får dermed ingen effekt på A. B blir derimot oppdatert av T2 først, og blir dermed berørt av både T1 og T2. Låser alene løser ikke alle samtidsproblemer.

To-faselåsing



I det ikke-serialiserbare forløpet oppdaterer begge flere elementer og setter og opphever låser underveis. Hvis alle transaksjoner ba om låser på samtlige elementer i starten av sitt forløp og opphevet samtlige låser til slutt ville ikke problemene oppstått. To-faselåsing bygger på dette prinsippet. En transaksjon med to-faselås får ikke lov til å sette nye låser etter at den første låsen er opphevet. Det betyr at livsløpet til en transaksjon kan deles inn i en voksefase og minkefase. I voksefasen blir nye låser satt. Når den første låsen blir opphevet går transaksjonen over i minkefasen. Da får den ikke lov til å sette på nye låser.

Hvis samtlige transaksjoner følger to-faselåsing får vi serialiserbare forløp.

Vranglåsing

T1	A	B	T2
Skrivelås A	10	10	Skrivelås B
Tell ned A med 10	0	20	Øk B med 10 hvis større enn 0
Skrivelås B?	0	20	Skrivelås A?
Vent	0	20	**Vent**
Vent	0	20	**Vent**
...	0	20	...

tid

I illustrasjonen til høyre har vi to transaksjoner som begge skal oppdatere A og B, og begge følger reglene for to-faselåsing. Transaksjon T1 setter lås på A, samtidig som transaksjon T2 setter lås på B. Så prøver T1 å sette lås på B, men må vente til B er låst opp av T2. Transaksjonen til T1 blir satt i ventekø. Samtidig prøver T2 å sette lås på A, men må vente på T1 og havner i ventekø. Vi har da to transaksjoner som venter på hverandre. Det har oppstått en *vranglås* (deadlock).

DBHS kan bruke to strategier for å løse vranglås:

1. Forhindre at vranglås oppstår
2. Oppdage og løse opp vranglås.

For å forhindre vranglås kan man bruke ***tidsstempler***. Systemet merker enhver transaksjon med tidspunktet den ble startet på, så ingen får nøyaktig samme tidsstempel. Når man får innvilget lås blir tidsstempelet lagret sammen med låsen. På denne måten kan systemet gi prioritet til den som var først og avbryte den som kom sist inn i vranglåsen. Da vil aldri en yngre transaksjon måtte vente på en eldre og vranglås kan ikke oppstå.

Alternativt kan man låse de opp etter de har oppstått. Da vedlikeholder man en ***ventegraf*:**

- Bygge opp en ventegraf: Hvis transaksjon T1 må vente på transaksjon T2, så legg til en kant (pil) fra T1 til T2.
- Av og til: Gå gjennom ventegrafen og sjekk om det har oppstått en sykel, for eksempel at T1 venter på T2 som venter på T3 som igjen venter på T1.
- Velg en av transaksjonene i cykelen. Avbryt transaksjonen («rull den tilbake»). Gjennomfør de andre, og start avbrutt transaksjon etterpå.

Isolasjonsnivåer

Serialiserbare transaksjoner kan være teoretisk bra, men i praksis kan det være behov for å løsne på krav av effektiviseringshensyn. Altså godtar man en viss innblanding fra andre transaksjoner. ***Usikker lesing***(dirty read) er at transaksjon T1 kan lese data til databasen av T2 før T2 har bekreftet med COMMIT. Da kan resultatet bli angret oppdatering. ***Ikke-repeterbar lesing*** (nonrepeatable read) betyr at T1 kan få to forskjellige svar på samme spørring fordi T2 har endret databasen og bekreftet med COMMIT. ***Fantomer*** (phantoms) er nye rader satt inn av T2 og som T1 oppdager når T2 bekrefter med COMMIT.

SQL definerer fire isolasjonsområder:

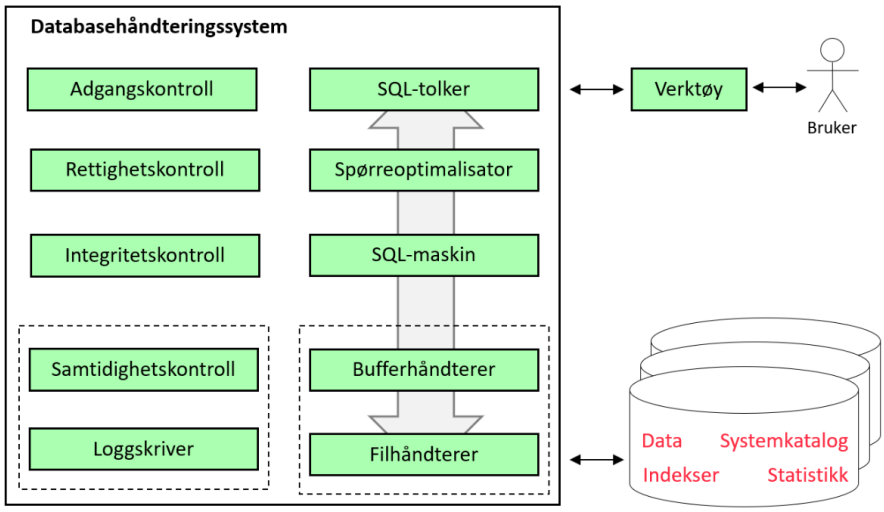
Isolasjonsnivå	Fantomer	Ikke-repeterbar lesing	Usikker lesing
SERIALIZABLE	Nei	Nei	Nei
REPEATABLE READ	Ja	Nei	Nei
READ COMMITTED	Ja	Ja	Nei
READ UNCOMMITTED	Ja	Ja	Ja

Pessimistisk og optimistisk låsing

Pessimistisk låsing er den vanlige typen lås. Vi har også noe som heter optimistisk låsing. Det betyr at en transaksjon alltid får lov til å begynne arbeidet med en rad. Kontrollen med eventuelle konflikter gjøres først når oppdaterte data skal lagres i databasen. Hovedregelen er at en transaksjon T1 får lagre en rad hvis ingen andre transaksjoner har oppdatert denne raden i tiden etter at T1 først leste raden.

Det er hensiktsmessig i systemer med få konflikter. For eksempel hvis de fleste kun leser, eller i interaktive databaseapplikasjoner. Transaksjoner blir utført uten restriksjoner fram til COMMIT men skriver til lokal kopi. Den lokale kopien blir validert før den skrives til databasen.

Databaseadministrasjon



DBHS brukes for å opprette nye databaser for jobbe med innhold i tabeller og for å administrere databaser. Et DBHS inneholder funksjoner for sikker lagring av store mengder data over tid, effektiv utførelse av oppdatering og søk fra mange samtidige brukere.

En databaseadministrator (DBA) har et overordna ansvar for databasesystemet. De viktigste oppgavene til en DBA etter databasesystemet er satt i produksjon er blant annet brukeradministrasjon, overvåking, sikkerhetskopiering, gjennoppbygging og optimalisering.

Oppgavene til DBA

Vi deler opp i databaseadministrator (DBA) og dataadministrator (DA).

En DBA har det overordna ansvaret for installasjon og drift av databasesystemer. DBA-stillingen innebærer en teknisk og en strategisk del. Fra strategiske beslutninger om arkitetktur, plattform, verktøy til tekniske oppgaver som sikkerhetskopiering. Oppgavene er ofte mer tekniske.

En DA har i oppgave å ha overordnet ansvar for databasedesign, prosedyrer og standarder knytta til datamodellering. Oppgavene er gjerne mer strategiske.

De viktigste oppgavene til DBA er:

- Valg, installasjon og oppgradering av DBHS
- Definere retningslinjer og prosedyrer
- Fysisk databasedesign
- Kapasitetsplanlegging og optimalisering
- Sikkerhetskopiering og gjenoppbygging etter feil
- Brukeradministrasjon
- Overvåking
- Opplæring og brukerstøtte

DBA-verktøy

Mange av DBA-oppgavene kan gjøres med SQL, men det finnes også egne verktøy for DBA. Deres typiske funksjonalitet er

- Starte og ta ned databasen
- Sikkerhetskopiering og gjenoppbygging etter feil
- Definere tabeller, indekser, osv.
- Brukeradministrasjon
- Sette grenser for brukernes ressursbruk
- Organisere fysiske lagringsstrukturer
- Skrive og analysere SQL-kommandoer
- Starte batch-jobber
- Visualisere bruk av disk, buffer
- Gjøre databasen tilgjengelig i et nettverk

Et DBA-verktøy som brukes er MySQL Server Administration som har disse funksjonalitetene.

Sikkerhetstrusler og virkemidler

- Uhell
  - Menneskelig svikt
  - Programvarefeil
  - Maskinvarefeil
  - Strømstans
- Angrep
  - Ansatte / eksterne
  - Innsyn / endring
  - Direkte mot databasen
  - Indirekte via operativsystem / nettverk

- Virkemidler
  - Sikkerhetskopiering, loggføring og gjenoppbygging
  - Overvåking
  - Kryptering
  - Aksesskontroll
    - Fysisk kontroll
    - Passord
    - Rettigheter
  - Duplisering av utstyr
  - Nødaggregat
  - Rutiner, «brannøvelser»

Sikkerhetskopiering og gjenoppbygging

Sikkerhetskopiering (backup) og gjenoppbygging (recovery) etter feil er en viktig oppgave for DBA. Uten sikkerhetskopiering av database vil diskkræsj gjøre at vi mister all data tilbake til databasen ble satt i drift. Derfor er det viktig med en solid opplegg for sikkerhetskopiering.

Fullstendig/inkrementell sikkerhetskopiering

Vi skiller mellom ***fullstendig sikkerhetskopiering*** der hele databasen kopieres til et annet lagringsmedium, og ***inkrementell sikkerhetskopi*** hvor det er kun det som er endret siden forrige gang som blir kopiert. Disse to kan kombineres feks ved at det hver natt blir tatt en inkrementell sikkerhetskopi og at det hver uke blir tatt fullstendig sikkerhetskopi. Man kan også ta en ***delvis sikkerhetskopi*** hvor kun deler blir kopiert, som feks noen tabeller.

Varm/kald sikkerhetskopiering

Ved ***kald sikkerhetskopiering*** må alle brukere logge seg av og databasen tas ned før sikkerhetskopi kan starte. Sikkerhetskopi blir av den grunn tatt på tidspunkter hvor det er lite aktivitet. ***Varm sikkerhetskopiering*** er da det motsatte, hvor sikkerhetskopi blir tatt mens databasen er i bruk. Disse løsningene kan være nødvendig for systemer som alltid må være tilgjengelige, men det gjør gjenoppbygging mer komplisert fordi databasen blir oppdatert mens sikkerhetskopiering pågår.

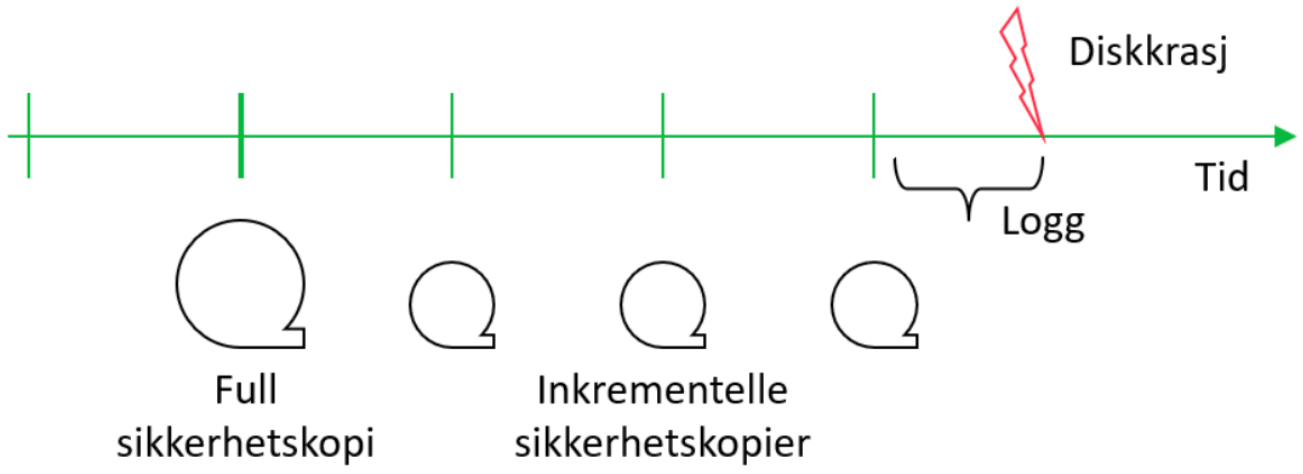
Fysisk/logisk sikkerhetskopiering

Databasetabeller blir lagret som filer på maskinen. ***Fysisk sikkerhetskopiering*** er basert på disse filene. ***Logisk sikkerhetskopiering*** eksporterer data som SQL-setninger. Førstnevnte tar mindre plass og vil få databasen raskere opp etter en feil. Sistnevnte brukes gjerne for mindre datavolum, men har den fordelen at formatet er uavhengig av maskintype og databaseversjon. Fordi sikkerhetskopi krever stor lagringskapasitet er det vanlig å komprimere data før lagring.

Det er egne verktøy for sikkerhetskopiering og gjenoppbygging. RMAN fra Oracle er et eksempel på dette.

Transaksjonslogg blir brukt til gjenoppbygging. Oppdateringer (transaksjoner) skrives først til transaksjonsloggen før det skrives til databasen. På denne måten kan man bruke siste sikkerhetskopi sammen med transaksjonsloggen for å få databasen tilbake i en konsistent tilstand så nærme feiltidspunktet som mulig. Etter at siste sikkerhetskopi er lastet inn (restored) må alle bekreftede transaksjoner kjøres på nytt fra transaksjonsloggen (recovery).

Sikkerhetskopier blir gjerne lagret på disker eller magnetbånd, oppbevart fysisk skilt fra selve databasemaskinen. Stadig flere bedrifter velger sikkerhetskopiering i skyen for nettopp dette.



Effektivitet

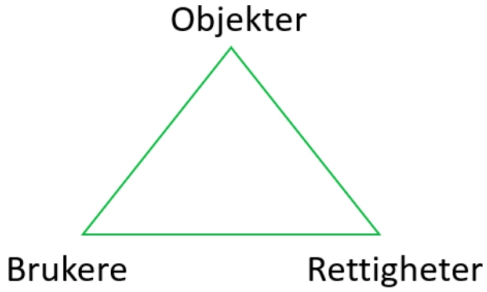
Effektivitet av en database er viktig. Desto større databasen er, desto viktigere er dette. Noen ting å ta hensyn til:

- Valg av maskinvare
  - Antall disker
  - Størrelse på hurtigminne
  - Hastighet på nettverk
- Fysisk design
  - Velge indekser
  - Denormalisering
  - Filstrukturer
- Overvåking og optimalisering
  - Skrive om SQL-kode, gi DBHS hint
  - Legge til indekser, endre fysisk lagringsstruktur
  - Endre systemparametre

Brukeradministrasjon

Brukeradministrasjon innebærer å opprette brukerkontoer med brukernavn og passord, og tildele brukere rettigheter i databasen. Deling av data er et hovedpoeng med databasesystemer. Likevel kan det være nødvendig/hensiktsmessig å begrense tilgang til visse data. Det kan feks være sensitiv data som kun noen få skal ha tilgang til. En annen viktig grunn er å sørge for at ikke alle har tilgang til å oppdatere data, noe som gjør det vanskeligere å sørge for korrekthet i databasen.

Brukeradministrasjon innebærer å definere forhold mellom brukere, rettigheter og databaseobjekter. Databaseobjekter kan være en tabell, en spørring, en visning, lagrede data, mm.



Rettigheter er tilgangen til grunnleggende operasjoner på databaseobjekter. For en tabell opererer man med leserett, rett til å endre, slette, lese definisjon, endre definisjon, slette hele tabellen, osv.

Vi skiller mellom ***objektrettigheter*** og ***systemrettigheter***. Systemrettigheter gir mer omfattende rettigheter for hele systemet, slik som å starte og stoppe et DBHS.

Brukerinformasjonen blir lagret i en eller flere systemtabeller.

Roller/grupper

En rolle (eller gruppe) er en samling brukere som skal ha de samme rettighetene i databasystemet. Som for eksempel rollen Selger. Roller gjør brukeradministrasjon enklere. Rollen blir som en abstrakt bruker

som tildeles rettigheter på samme måte som en bruker. Så kan man tildele brukere en rolle. Da slipper man å endre rettigheter på mange enkeltbrukere, men kan istedenfor endre rettighetene til en rolle. Brukerne arver rettighetene til rollen.

Sikkerhetssystem og brukere

Vi skiller mellom noen brukertyper:

– Administratorer: Får alle rettigheter, også systemrettigheter som å opprette andre brukere, starte og stoppe databasen, lage tabeller osv.

– Endre: Sluttbrukere som både kan avlese og endre innholdet av databasen. Får rettigheter til å avlese / sette inn / oppdatere og slette data (SELECT + INSERT + UPDATE + DELETE).

– Innsyn: Får rettigheter til å avlese innholdet i databasen (SELECT).

– Utviklere: Jobber med strukturen. Får rettigheter til å lage nye tabeller, spørringer, utsnitt, og også å endre definisjonen av eksisterende objekter. Jobber vanligvis ikke med databaser i produksjon.

- Forskjellige deler av databasen blir brukt av forskjellige avdelinger/stillingskategorier.
- Vi deler inn gruppen av sluttbrukere. Eksempler:
  - AvdelingA, AvdelingB, …
  - Sekretær, Selger, Direktør, …
  - Nivå1, Nivå2, Nivå3, …
- Kan være nyttig å lage roller i flere «lag».
  - I Oracle kan roller tildeles andre roller. Eksempel: Både rollen Selger og rollen Sekretær trenger rollen Innsyn.
  - Hvis Selger og Sekretær skal ha mange av de samme rettighetene er det arbeidsbesparende å tildele rettighetene 1 gang til Innsyn, og deretter gi Innsyn til Selger og Sekretær.
- Kan tildele rettigheter på views som gir tilgang på utvalgte rader/kolonner i en tabell.

Overvåking

For at rettighetssystemet skal fungere etter hensikten er det nødvendig med tiltak. Man bør velge gode passord og lære opp brukere til sikker bruk. Det kan være lurt å innføre fysisk adgangskontroll til maskinparken. Man vil unngå at uvedkommende bryter seg inn i systemet eller at brukere går utover sine tildelte rettigheter. Man kan overvåke bruken av databasen. Med kommandoen AUDIT kan man overvåke brukere. Man kan også ha triggere som loggfører når en hendelse inntreffer og hvem som gjorde den.

Systemkatalogen

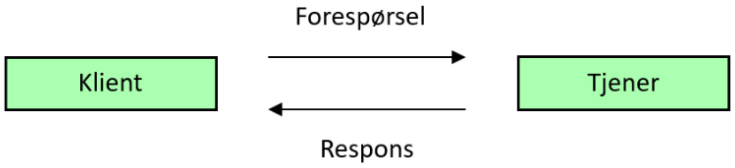
Systemkatalogen er en samlebetegnelse av denne typen data:

- ***Metadata,*** eller «data om data» er beskrivelser av databasen:
  - Hvilke tabeller finnes? Hvordan er tabellene bygget opp?
  - Hvilke indekser er definert?
- ***Statistiske data*** brukes av spørreoptimalisatoren:
  - Hvor mange rader har tabellene?
  - Hvor mange forskjellige verdier er lagret i en kolonne?
- Data om ***brukere og deres rettigheter***.

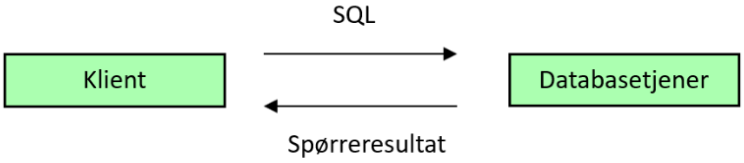
Data i systemkatalogen er ofte lagret på tabellform (systemtabeller), som betyr at vi kan bruke SQL for rapportering.

Klient/tjener-arkitektur

- Generelt



- Databaser



- Klienter og tjener i nettverk

Skytjenester

Skytjenester innebærer å gi tilgang på maskiner, nettverk og programvare over internett. Man kan abonnere på ulike tjenester istedenfor å etablere egne løsninger. En fordel med dette kan feks være skalering. Ved egen drift må man kjøpe inn maskinvare og utstyr som kan bli dyrt å skalere. Et DBHS kan være komplekst å administrere, men ved hjelp av skytjenester blir det et redusert behov for teknisk kompetanse. Man betaler ofte for bruk som gjør at man gjerne betaler for faktisk bruk.

IaaS (Infrastructure as a Service) er en basistjeneste der man får tilgang på en eller flere maskiner, men må selv installere operativsystem og programvare.
Eksempel: AmazonWeb Services.

PaaS (Platform as a Service) er en mer omfattende tjeneste enn IaaS, der man i tillegg til maskiner og operativsystem også får et ferdig utviklingsmiljø i skyen, f.eks. i form av en Linux-tjener satt opp med en MySQL database.

SaaS (Software as a Service) er en ytterligere utvidet tjeneste i forhold til PaaS og kan betraktes som en måte å tilby programvare på.
Eksempel: Google Apps og Office365.

Skydatabase (cloud database) er en database som blir lagret i skyen, og kan i utgangspunktet være basert på både IaaS, PaaS og SaaS.

DBaaS (DataBase as a Service) er en SaaS-løsning for databaser, der man typisk både oppretter, administrerer og bruker databasen via en webapplikasjon.

Azure SQL Database er enkelt sagt «SQL Server i skyen» og kan fås som en DbaaS-løsning.

Spørreoptimalisering

Spørreoptimalisering innebører å finne en effektiv måte å utføre SQL-spørringer på. Optimalisering er et litt misvisende ord, for det er ikke alltid det er den mest effektive måten. For å analysere SQL-spørringer bruker man relasjonsalgebra.

Et DBHS analyserer SQL-koden for å finne den mest effektive rekkefølgen å utføre deloperasjoner på. Dette blir kalt spørreoptimalisering. Det deles inn i to faser:

- Oversette SQL-setningen til et uttrykk i relasjonsalgebra
- Omskriving av uttrykkene til en form som lar seg utføre mer effektivt.

<u>Eksempel på omskrivningsregler:</u>

– Skyv seleksjoner og projeksjoner «innover» til de tabellene de gjelder.
– Bytt om på likekoblinger for å minske størrelsen på mellomresultater.
– Bryt ned sammensatte seleksjoner i flere enkle.

Kostnadsbasert optimalisering utnytter statistiske data om databasen for å optimalisere.

– Bruker omskrivningsregler for å generere et antall planer – tildeler hver plan en kostnad basert på statistikk om bla. størrelsen på tabeller – og velger den beste.

Lagrede rutiner og triggere

En lagret rutine er enten en lagret funksjon eller en lagret prosedyre og kan brukes av klientapplikasjoner som feks et PHP-skript.

En trigger er et delprogram som blir utført automatisk når visse hendelser inntreffer. Feks ved hver innsettning i en bestemt tabell.

Lagrede rutiner

En lagret rutine kan være en lagret prosedyre eller en lagret funksjon. Slike rutiner blir lagret og utført på databasetjeneren. De kan kalles fra flere klienter. De kan både inneholde grunnleggende SQL-kommander som INSERT og steg-for-steg kode med variabler, valg og løkker som i generelle programmeringsspråk. De kan brukes til automatisering og hjelpe med gjenbruk av kode. Dette er med på å effektivisere og redusere nettverkstrafikk. Det er vanlig å lagre de lagrede rutinene i egne scriptfiler.

Ulike lagrede rutiner:

Lagret prosedyre

- Registrere ny ansatt
- Data om den ansatte er parametre
- Kontrollerer inndata og setter inn ny rad hvis alt er ok
- Kan sende tilbake eventuell feilmelding i en ut-parameter

Lagret funksjon

- Gjennomsnittspris for varer i en kategori
- Kategorinummer er parameter
- Prisen blir returverdi

Lagret rutine

- Funksjon **eller** prosedyre

Redefinere skilletegn

Vi bruker semikolon til å avslutte en kommando og vi har i en lagret rutine gjerne flere kommandoer som igjen avsluttes med semikolon. For å kunne bruke skilletegn kan vi redefinere semikolonet ved hjelp av DELIMITER. Vi kan foreksempel starte setningen med DELIMITER $$ og avslutte med DELIMITER ; for å redefinere semikolon til $$

Cursors (markører)

En cursor er en peker som refererer til en rad i spørreresultatet. Vi kan for eksempel bruke en peker i en løkke til å løpe gjennom hver rad, som da flyttes nedover. Da får vi alltid verdier fra en rad av gangen.

Deklarerer cursoren

```
CURSOR c1 IS SELECT snr, navn FROM student;
```

Åpner cursoren

```
OPEN c1;
```

Henter én og én rad i en løkke til det ikke er flere rader

```
LOOP
  FETCH c1 INTO v_snr, v_navn;
  -- Hopp ut av løkka hvis forbi siste rad …
END LOOP;
```

Lukker cursoren

```
CLOSE c1;
```

3 typer parameteroverføring

IN - parametre
- Er standard og svarer til verdioverføring i Java/C
- Benyttes for å sende verdier inn til prosedyren

OUT - parametre
- Benyttes for å returnere verdier fra prosedyren
- Hensiktsmessig hvis man vil returnere mer enn én verdi

IN OUT er en kombinasjon av IN og OUT
- Variabelverdien sendes inn i prosedyren og kan endres der
- Ev. endret verdi sendes tilbake fra prosedyren

Merk:
- Variable som IN-parametre endrer **ikke** verdi som følge av prosedyrekallet
- Variable som OUT og IN OUT parametre **kan** endre verdi

Triggere

Triggere er et (lite) program som blir utført i databasen

- Kjøres automatisk som følge av en hendelse i databasen
- Mulige hendelser: INSERT, UPDATE, DELETE
- Vi sier at triggeren avfyres

Triggere er små programsnutter på databaseserveren som settes i gang før eller etter en hendelse som insert, update og delete. Programmene kan kjøres før endringer skrives til databasen eller etter at det er skrevet endringer til systemet. En trigger benyttes for operasjoner som skal gjøres uten unntak. Triggere kan f.eks benyttes til å validere inndata eller til å loggføre etter at en operasjon har funnet sted.

Vi har to hovedtyper av triggere BEFORE triggere og AFTER triggere.

En BEFORE trigger fyres av før oppdateringen skjer i databasen og kan benyttes til:
- Automatisk formatering av verdier
- Automatisk datostempling av verdier
- Kontroll ved forretningsregler
- Finmasket tilgangskontroll (feks forhindre vise-operasjoner utenfor arbeidstid)
- Automatisk utfylling

En AFTER trigger fyres av etter oppdateringen har skjedd i databasen og brukes typisk til å:

- Automatisk oppdatere data i andre tabeller
- Automatisk loggføre endringer
- Oppdatere avledede data i andre tabeller.
- Samle statistiske data
- Implementere denormalisering

```
CREATE TRIGGER triggernavn
  [BEFORE | AFTER]
  [INSERT | UPDATE | DELETE] ON tabellnavn
  [FOR EACH ROW]
BEGIN
 setninger
END
```

Ett eksempel på en kreasjon av en trigger

```
CREATE TRIGGER tr_Ansatt
     BEFORE UPDATE ON Ansatt
     FOR EACH ROW
     BEGIN
     set new.Fornavn = UPPER(new.Fornavn);
END
```

Kan benyttes til loggføring, automatisk følgeoppdatering og håndtering av forretningsregler.

Rad-trigger

En rad-trigger blir avfyrt for hver rad som blir berørt av en oppdatering. Feks:

```
CREATE TRIGGER tr_ansatt_1
        BEFORE UPDATE ON ansatt
        FOR EACH ROW
BEGIN
        -- Behandle raden
END
```

Vi har også noe som heter setnings-triggere, som avfyres for hver SQL-setning. Disse støttes ikke i MySQL.

OLD og NEW

To innebygde «postvariabler» OLD og NEW

- OLD → opprinnelige verdier før endring
- NEW → nye verdier slik de **vil bli** etter oppdatering i databasen
- Ved INSERT er OLD = NULL, ved DELETE er NEW = NULL

Hvordan referere til data i raden som oppdateres

- OLD.<kolonnenavn> gir gammel verdi
- NEW.<kolonnenavn> gir ny verdi

BEFORE triggere → kjøres <u>før</u> databasen oppdateres

- NEW-verdier er endret og tilgjengelige selv om databasen ikke er oppdatert ennå!
- Triggeren kan endre NEW-verdier **før** oppdatering i basen
- Automatisk konvertering/formatering av data som settes inn
- Automatisk oppdaterte dataverdier, f.eks. datostempel som viser når raden sist ble endret

AFTER triggere → kjøres <u>etter</u> at databasen er oppdatert

- Da er det **for seint** å endre NEW-verdiene.
- OLD og NEW-verdier kan fremdeles **refereres**, f.eks. for å oppdatere andre tabeller

Hva inneholder bufferne?

	OLD	**NEW**
INSERT	NULL i alle kolonner	Verdier i den nye raden som skal settes inn
UPDATE	Verdier slik de er/var i databasen før oppdatering	Verdier slik de vil bli i databasen etter oppdatering
DELETE	Verdier slik de er/var i databasen før sletting	NULL i alle kolonner

Rad-triggere

	BEFORE	**AFTER**
INSERT	Ny rad er ikke satt inn ennå	Ny rad er satt inn i tabellen
UPDATE	Raden er ikke oppdatert i tabellen	NEW-verdier er oppdatert i tabellen
DELETE	Raden er ikke slettet ennå	Raden er slettet

(Setnings-triggere – ikke støttet i MySQL)

	BEFORE	**AFTER**
INSERT	Ingen rader er satt inn ennå	Alle nye rader er satt inn i tabellen
UPDATE	Ingen rader er oppdatert ennå	Alle berørte rader er oppdatert i tabellen
DELETE	Ingen rader er slettet ennå	Alle rader som berøres av DELETE er slettet fra tabellen

Objekter

Relasjonsdatabaser egner seg mindre godt til å håndtere komplekse datastrukturer. Ta for eksempel tekniske tegninger, multimedia, semistrukturert data, kart, sosiale medier eller big data. I disse tilfellene gir det ofte ikke mening å bruke en relasjonsdatabase. Dette er hvor NoSQL ofte kommer inn i bildet. NoSQL står for «Not only SQL».

Objekter

Et objekt er data + metoder. Vi kan si at det er en «ting», eller et begrep. For eksempel Ansatt. Det kan sammenliknes med en entitetsforekomst i ER. På samme måte som entiteter har et objekt også attributter. I programmering tilsvarer dette variabler i et objekt.

Et objekt er til enhver tid i en bestemt tilstand. Feks AnsattNr = 1, Navn = Per, osv. Et objekt kan gå inn i en ny tilstand men er fortsatt det samme objektet. Et objekt har også metoder – enAnsatt.endreLønn(nyLønn)

Objekter og klasser

Vi skiller mellom objekter og klasser. Et objekt med fornavn Per, etternavn Hansen, etc. er eksempel på en instans (forekomst) av klassen Ansatt. En klasse kan sees på som en mal man kan danne objekter fra. En klasse beskriver hva slags data og hvilke metoder objektene i klassen skal inneholde. Alle objekter av en gitt klasse har samme struktur. Under programkjøring blir det generert objekter av klassene (instansiering). Dette er likt som i Java-programmering.

I en relasjonsdatabase har alle tabeller en primærnøkkel som identifiserer rader i tabellen. Tilsvarende har objekter en ***objektidentifikator (OID)***. Det er en viktig forskjell mellom rader i en relasjonsdatabase og objekter: Det er mulig å ha to forskjellige objekter med samme verdi for samtlige attributter (utenom OID).

Metoder og innkapsling

Til forskjell fra entiteter har objekter ***tilstand*** og ***oppførsel.*** Tilstanden til et objekt kan endre seg over tid. Selv om Ansatt-objektet Per skulle skifte etternavn, er det fremdeles snakk om det samme objektet. Objektet beholder sin OID gjennom hele livsløpet og det finnes ikke to objekter med samme OID.

Oppførselen til et objekt er definert ved metoder. Metodene inneholder programkode som jobber med data i objektet. Objekter kommuniserer ved å be andre objekter utføre operasjoner. Et objekt kaller metoder i andre objekter. En metode kan sammenliknes med en prosedyre eller funksjon.

Et objekt kan være beskyttet ved at andre objekter ikke har direkte tilgang til det. Da må man istedenfor kalle på objektmetodene. Det kalles innkapsling. I Java bruker vi gettere og settere til dette.

Arv og polymorfi

Objektorienterte språk (som Java) har også arv og polymorfi mellom objekter. Dette er på samme måte som subklasser og superklasser i ER-diagram. Vi får et klassehierarki/arvehierarki. Objekter kan arve attributter og metoder fra andre. Dette har vi jobbet med i programmering, så jeg går ikke nærmere inn på det her. Det står nevnt i boka.

Objektrelasjonelle databaser

Ny bruk av databaser, som feks kartdata, førte til forskning på alternativer til relasjonsmodellen. Vi fikk da ***objektdatabaser***. Da ble også relasjonsdatabaser utvidet med støtte for objektorienterte mekanismer. Som feks. Mulighet til å definere egne datatyper med egne metoder som kan brukes i tabeller og SQL, Collections klasser og subklasser. Disse hybridsystemene kalles objektrelasjonelle databaser. Disse ble mer populære en rene objektdatabaser.

Relasjonsmodellen krever atomære verdier. Et sett av datatyper er forhåndsdefinert og alle verdier må tilhøre en av disse datatypene. Vi har tenkt på en databasetabell som en todimensjonal struktur organisert i rader og kolonner. Mna kan imidlertid tenke på en rad som et objekt. Databasetabeller blir i så fall endimensjonale samlinger av objekter. UML er noe som ofte blir brukt i objektrelasjonelle databasser. UML er et objektorientert språk som kan brukes for å beskrive både datamodeller, applikasjoner og fysisk organisering på maskiner og nettverk. (UML har ikke vært pensum i år).

I webprogrammering har vi brukt JDBC (Java DataBase Connectivity) til å jobbe med databaser. Dette er et ferdiglaget klassebibliotek som gjør at vi kan jobbe med databaser fra Java. JDBC er et verktøy som pakker inn relasjoner som objekter mot en relasjonsdatabase.

NoSQL

NoSQL er en samlebetegnelse på en rekke nyere databasesystemer som organiserer data på andre måter enn i tabeller. NoSQL betyr «Not Only SQL). Noe av det som kjennetegner disse er håndtering av komplekse datastrukturer og semistrukturerte data, distribuerte løsninger og løsere krav til transaksjonshåndtering.

Big Data er noe som er avhengig av NoSQL-databaser. Vi har datasett som er så store og kompliserte at de tøyer grensene for hva som er mulig å håndtere, både med hensyn til lagring, utveksling og behandling.

- Volume: Vi lagrer langt mer data enn før. Bilder, videoer, værdata, osv.
- Velocity – Data blir generert i høyere tempo enn før. Feks. GPS-data, logging av teletrafikk, brukeraktivitet, vitenskapelige data, mm.
- Variety – Data er mer variert enn før. Ustrukturert data som dokumentsamlinger, mm. Dette passer ikke i relasjonsdatabasenes strenge tabellstruktur.
- Veracity – Hvorvidt data er presist og pålitelig. Data kan være upålitelig og upresist og usikkert.

Karikteristiske trekk ved NoSQL

Flere kjente NoSQL-databaser er utviklet av og for de største IT-bedriftene, men er også veldig aktuelle for brukere med mindre bedrifter. Databaser via skyløsninger gjør at man lett kan skalere opp lagringskapasitet og antall brukere raskt ved behov uten å måtte investere i en egen maskinpark. Dette kan være fint for mindre bedrifter. Man skiller gjerne mellom ***horisontal skalering*** som innebærer å utvide med flere maskiner og ***vertikal skalering*** hvor man oppgraderer til kraftigere maskiner.

Mange NoSQL-databaser egner seg godt i ***distribuerte løsninger***, gjerne fordelt på mange og enkle maskiner, der data både blir replikert og fragmentert. Altså man distribuerer data over på mange små, istedenfor en stor. Man fordeler arbeidsbelastning jevnt.

Relasjonsdatabaser krever at alt passer inn i tabeller. NoSQL-databaser er mer fleksible, på samme måte som objektdatabaser og XML-databaser. Typiske NoSQL-data er sammensatte og/eller semi-strukturerte. For eksempel en nettside bygd opp av overskrifter, avsnitt, lister men også uthevede ord, senker skrift, ulike fornter og skriftstørrelser.

NoSQL-databaser har støtte for spørrespråk, men det er gjerne snakk om språk som er enklere enn SQL. Typisk rettet inn mot enkle CRUD-operasjoner.

Relasjonsdatabaser tilbyr flerbruksløsninger med full støtte for transaksjoner med ACID-egenskaper. Klassisk tofase-låsing garanterer serialiserbare kjøreplaner som igjen gir en konsistent database. NoSQL derimot gir slipp på noen av transaksjonskravene.

CAP-teoremet sier at et distribuert system aldri kan garantere mer enn 2 av følgende 3 egenskaper:

- Consistency – På et bestemt tidspunkt ser databasen lik ut for alle noder. Altså, alle får alltid helt oppdaterte data
- Availability – Alle forespørsler får en respons (som også kan være en feilmelding)

- Partition tolerance – Systemet fortsetter å fungere selv om vilkårlig mange forbindelser mellom noder er brutt.

Ved å slippe litt opp på absolutt krav om konsistens kan man oppnå høyere tilgjengelighet. Det vil si at spørringer kan bli besvart hurtig, selv om deler av nettverket er nede. Noen SQL-databaser velger tilgjengelighet fremfor konsistens.

Et alternativ til ACID-egenskapene er ***BASE***-egenskapene: Basically Available, Soft-state, Eventually Consistent. Disse passer godt for distribuerte NoSQL-databaser, der oppdateringer blir kopiert utover nettverket. På grunn av forsinkelsen kan man risikere å lese ikke (helt) oppdaterte data.

NoSQL-familiene

NoSQL er en samlebetegnelse for mange ulike løsninger. Det er vanlig å dele disse inn i fire «familier»

Nøkkel/verdi-databaser (key/value)

Data er organisert ved en samling av nøkkel/verdi par som feks fornavn='Ola'. Slike databaser er velegnet for applikasjoner som gjør mange, enkle oppslag basert på nøkler. Riak, BigTable, Redis og Oracle BerkeleyDB er noen eksempler på disse

Dokumentdatabaser

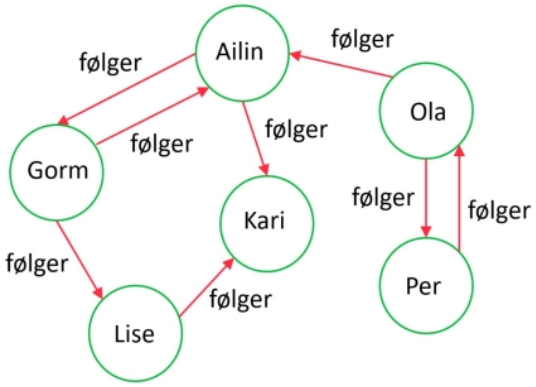
Databasen består av semi-strukturerte dokumenter. Dokumentene kan være representert i form av XML, JSON eller lignende formater, og hvert dokument har en unik id (nøkkel). De er velegnet som database bak web-løsninger, og tilbyr gjerne et spørrespråk for å hente ut dokumenter basert på innhold. MongoDB, CouchDB og CouchBase er noen eksempler på disse

Kolonnedatabaser

Data blir lagret som nøkkel/verdi-par, men ulike kolonner (eller kolonnefamilier) blir lagret hver for seg. Tabeller i en relasjonsdatabase inneholder ofte mange kolonner, men typiske spørringer bruker kun noen få av disse. Ved å splitte opp tabeller i kolonner (som hører sammen) kan man oppnå en effektivitetsgevinst. HBase og Cassandra er eksempler på disse.

Grafdatabaser

Data er representert som en graf (nettverk) bygd opp fra noder og kanter. Slike databaser egner seg for domener som er naturlig grafaktige, som feks sosiale nettverk – Hvilke personer følger hvilke andre på Twitter. Neo4J, Titan og Microsoft Azure Cosmos DB er eksempler på disse.



Databasehistorien startet med hierarkiske databaser og nettverksdatabaser tilbake på 1960-tallet. Edgar F. Codd argumenterte mot denne måten å organisere data på, fordi den var basert på navigasjonsstrukturer, eller bruk av «pekere». Med objektrelasjonelle databaser og NoSQL-databaser jobber vi igjen med «pekere». Spørringene vi skriver mot slike databaser må følge disse pekerne, og blir dermed avhengige av den valgte datastrukturen.

Oppsummert: Relasjonsdatabaser egner seg godt for strukturerte data, men ulike NoSQL-databaser egner seg bedre for anvendelser som må håndtere komplekse og/eller semistrukturerte data

Torunn sin oppsummering av NoSQL:

Fordeler med relasjonsdatabaser:

- Sjekk for integritet og referentiell integritet gir kvalitet på data.
- Støtte for transaksjoner.
- Autentisering og kontroll av rettigheter ved brukt av systemet.
- Logging av data og muligheter for recovery om noe går galt.
- Spørringer kan få ut alle mulige kombinasjoner informasjon fra tabellene i systemet
- SQL er et felles språk.
- Følger ACID egenskaper.

Ulemper med relasjonsdatabaser:

- Store ressurskrevende dataprogrammer
- Data lagret i mange tabeller fordelt på ulike steder på lagringsmediet gir trege systemer.
- All kontroll av rettigheter og alt tilleggsarbeid systemet utfører gjør det å hente data tregere.

NoSQL databasesystemer er en sekkebetegnelse på mange ulike databasesystemer som deles i følgende grupper:

- Nøkkel/verdi-databaser
- Dokumentdatabaser
- Kolonnedatabaser

Felles for systemene er at datamengdene ofte er store, krav til skalering for mange brukere, krav til hurtig levering av data. Noen NoSQL databasesystemer av egner seg til sanntids informasjonsdeling feks sosiale medier. Det kan være sanntidssystemer som spilldata som replikeres til brukerne, det kan være store datamengder som registreres automatisk og bare aggregeringsdata som skal lagres varig. Systemene kan være dokumentbaserte og er laget for at informasjonsinnhenting skal gå fort, dette kan være i XML eller i JSON format. Databasen ligger ofte på flere maskiner og data blir replikert og fragmentert. Data i NoSQL systemer er ofte mindre strukturerte enn de vi har i relasjonsdatabaser. Ulemper er at man må sette seg inn i nye APIer og lære seg nye måter å få data ut av databasene

Hvis integritet og sikkerhet er viktig som f.eks i et banksystem ville det være nødvendig å benytte relasjonsdatabaser. Facebook-data er helt annerledes. Her er datamengdene enorme og krav til skalering og hurtighet veldig store. I et slikt system ville det være nyttig å benytte en NoSQL-løsning

XML

Vi har ikke gjennomgått dette, så det kommer nok ikke på eksamen

XML er spesielt velegnet for datautveksling, der databaser er best for datalagring. Dette kan for eksempel være data som benyttes av andre applikasjoner (web). XML har en hierarkisk sortering av data og har en mer fleksibel struktur enn en database. XML burde ikke benyttes til lagring av store mengder data. XML er egnet som format på web fordi det er rene tekstfiler som gjør at det i utgangspunktet er uavhengig av programvare eller system. HTML som er det vanligste formatet å presentere innhold på web er definert i XML. Siden XML krever hierarkisk oppbygging av dataene, er det mulig å veldefinere strukturen til websiden. I XML lager vi en tre-struktur av databasetabellen: Ett element for hele tabellen, ett element for hver rad og ett element for hvert felt

XML kan med fordel benyttes når:

- Data skal deles med andre
- Data skal leses, men ikke slettes og endres
- Det ikke skal gjøres kompliserte spørringer mot dataene
- Data ikke trenger tilgangskontroll
- Data skal utveksles mellom ulike systemer
- Data skal lagres uavhengig av datasystemer
- Data skal leses og presenteres på web-side

XML er format som det er støtte for i de fleste browsere. Hva er XML og hvorfor er det egnet som format på web?

XML er et metaspråk, et språk for å definere nye språk. Det står for eXtensible Markup Language, og man kan definere egne tagger og strukter i XML-tekstfiler. XML defineres ofte av en DTD-fil eller Document Type Definition som definerer strukturen av taggene, om taggene er obligatoriske og hva slags datatype de er. XML Schema kan også brukes som Dokument Definisjon for XML-filer, som er et rikere DTD med flere funksjoner som DTD ikke støtte, som feks Støtte for nøkler, flere datatyper, og støtte for XML navnerom. XML schema bruker også XML syntaks i motsetning til DTD.

Det er godt egnet som format på web fordi det er rene tekstfiler, som gjør at det i utgangspunktet er programvare og system-uavhengig. HTML som er det vanligste formatet å presentere innhold på web er definert i XML. Siden XML er rene tekstfiler er de ideele for datautveksling.

Siden XML krever hiarkisk oppbygging av dataene, er det mulig å veldefinere strukturen til websiden. XML lar deg også programmere mot taggene og få mange typer data ut ved bruk av XSLT.

SQL-spørringer

Se også eget SQL-oppslagsverk

Generelle

• Create table – husk primær og fremmednøkler

• Insert, update, delete

• Drop table

• Right join, left join

• Select

• Sum,max,min husk group by

• Order by, limit 1

Tabeller

Opprette tabell

```
CREATE TABLE Ordre(
	OrdreNr INTEGER AUTO_INCREMENT,
	OrdreDato DATE,
	Beskrivelse VARCHAR(255)
	KNr integer FOREIGN KEY REFERENCES Kunde(KNr),
	AnsNr INTEGER FOREIGN KEY REFERENCES Ansatt(AnsNr),
	SendtDato DATE,
	BetaltDato DATE,
	PRIMARY KEY (OrdreNr)
);
```

Sette inn i tabell

```
INSERT INTO Bok(ISBN, Tittel, Forfatter, Forlag, UttgittÅr, AntallSider)
VALUES(12345678, 'Jernblod', 'Hansen', 'Kagge','2015', 267);
```

Nestede spørringer

Opprettelse

Database/tabell

Databaseadministrasjon

Brukerrettigheter
CREATE DATABASE
CREATE USER
GRANT (rettigheter) ON (objekt) TO (user);

Prosedyrer (lagrede rutiner)

Transaksjoner

```
START TRANSACTION;
        INSERT INTO Ordre(OrdreNr,Knr,AnsNr)
        VALUES (2020,1003,22);
        INSERT INTO Ordrelinje(OrdreNr,Vnr,Antall)
        VALUES (2020,'4034',5);
        UPDATE Vare
        SET Antall = Antall-5
        WHERE VareNr='4034';
COMMIT;
```

```
SET autocommit = 0;
```

Triggere

```
CREATE TRIGGER triggernavn
        [BEFORE | AFTER]
        [INSERT | UPDATE | DELETE] ON tabellnavn
        [FOR EACH ROW]
BEGIN
        setninger
END
```

```
CREATE TRIGGER tr_Ansatt
	BEFORE UPDATE ON Ansatt
	FOR EACH ROW
BEGIN
	set new.Fornavn = UPPER(new.Fornavn);
END
```

Indeksering

```
CREATE INDEX VarenavnIdx
ON Vare(Varenavn);
```

```
CREATE UNIQUE INDEX EtternavnIdx
ON Ansatt(Etternavn);
```

Metadata

<u>Brukerdefinerte datatyper</u>

```
CREATE TYPE AdresseType AS
(
	Gate VARCHAR(50),
	Nummer INTEGER
);
```

```
INSERT INTO Student(Id, Fornavn, Adresse)
VALUES (5, 'Ole', ROW('Furulia', 28));
```

```
SELECT Id, Fornavn, (Adresse).Gate
FROM Student;
```